

01

AI Engineering
Environment Setup

The complete reference for building, debugging
and maintaining a working AI development machine
— on any operating system, for the length of
a career.

OBJECTIVE	PLATFORMS	PYTHON	REVISION
Operational readiness for AI engineering workflows	macOS · Linux · WSL2 · PowerShell · Colab	3.11, pinned everywhere	1.0 — keep with your tools

Contents

FIVE PARTS · TWENTY-NINE CHAPTERS · THREE APPENDICES

PART I — GROUND

- 01 How to use this manual
- 02 Your machine: hardware, OS, and what it can actually run
- 03 The terminal: shells, commands, and the twenty you need
- 04 Files, paths, permissions and PATH itself

PART II — PYTHON

- 05 Python itself: interpreters, versions, and which one you are running
- 06 Virtual environments: the mental model
- 07 pip
- 08 conda and miniforge
- 09 uv
- 10 poetry
- 11 Choosing between them — and migrating
- 12 Dependency resolution: how it fails and how to read the failure
- 13 Reproducible environments: pinning, lockfiles, hashes

PART III — TOOLS

- 14 VS Code
- 15 Git: the model, then the commands
- 16 GitHub workflows: branches, pull requests, review
- 17 SSH keys, remote development, dev containers
- 18 Shell productivity: aliases, dotfiles, tmux
- 19 Makefiles, task runners, pre-commit hooks
- 20 Jupyter: notebook hygiene and notebook-to-script

PART IV — COMPUTE

- 21 Google Colab and free GPU
- 22 GPUs, drivers and CUDA: the compatibility matrix
- 23 Renting GPUs, and not being surprised by the bill
- 24 Docker, from first principles
- 25 Docker in practice: images, volumes, GPU containers
- 26 Model caches and disk hygiene

PART V — KEYS, COST, DISCIPLINE

- 27 API keys and secrets: storage, rotation, and what to do after a leak

- 28 Calling model APIs: rate limits, retries, token cost
- 29 Continuous integration with GitHub Actions

APPENDICES

- A Pre-flight checklists, one per platform
- B The day-one setup script, annotated line by line
- C Networks, proxies and firewalls: the reference chapter

01 PART I — GROUND How to use this manual

This is a manual, not a course. You will read Part I once, in order, on your first day. After that you will almost never read this book front to back again — you will arrive at it mid-problem, with an error message on your screen and twenty minutes before a deadline, and you will want the right page in under a minute. Everything about the way it is built serves that second use.

The three kinds of page

Explanations build the mental model. They are the pages worth reading before you need them, because they are what make the other two kinds obvious rather than magic. There are not many of them, and they are short.

Tables are for the middle of a problem. Every chapter ends with a symptom → cause → fix table. Read down the left column until you find your symptom, in your own words rather than the computer's. If your symptom is not there, chapter 12 teaches you how to read an error message you have never seen before, which is the skill this manual is really trying to install.

Labs are the only way any of this becomes yours. Each lab is one step of a single project — a command-line tool called `summarise-cli` that reads a text file and summarises it with a language model. It starts in chapter 06 as an empty folder. By chapter 29 it is a packaged, tested, containerised tool with pinned dependencies, a secret it does not leak, retries around its API calls, and continuous integration that catches you when you break it. Do the labs in order. Every one takes between five and thirty minutes.

Conventions

YOU SEE	IT MEANS
<code>\$ command</code>	Type this in a terminal on macOS, Linux or WSL2. Do not type the <code>\$</code> .
<code>PS> command</code>	Type this in Windows PowerShell.
<code>>>> code</code>	Type this inside a Python interpreter.
<code><your-name></code>	Replace the whole thing, angle brackets included, with your own value.
Golden rule	A rule that will save you hours. There are twenty-one of them and they are worth learning by heart.
Verify	A command that proves the step worked. Never move on without running it.
Ages fastest	End of each part: which advice in that part is most likely to be wrong by the time you re-read it.

Two assumptions this manual makes about you

First, that you may not have administrator rights on your machine. Every install in this book has a user-level path that needs no password, and where the admin path is genuinely different you get both, side by side. If a command in this manual asks you for `sudo` and you cannot give it, there is an alternative within a page.

Second, that your network may be hostile. Corporate proxies, TLS inspection and blocked ports break more first-day setups than any other cause, and they break them with error messages that blame the wrong thing entirely. If a download or a `git clone` fails for a reason you cannot explain, stop and go to Appendix C before you debug anything else.

GOLDEN RULE 01

Change one thing, then verify. Two changes at once means you no longer know which one worked, and you will spend the next hour finding out.

How to ask for help, so that you get it

You will be stuck often, and how you ask determines whether you are unstuck in two minutes or two days. A good request has five parts, in this order: what you were trying to do, the exact command you ran, the complete error message pasted as text rather than a photograph, your platform and Python version, and what you already tried. Copy the output of `python -V`, `which python` and `pip -V` into every request; roughly half of all environment questions are answered by those three lines alone.

Never paste a screenshot of text. Never paste the first line of an error and stop — the useful line is usually the last one. Never edit your error message to look tidier; the ugly detail is often the answer.

Reading order if you are in a hurry

1. Appendix A, the checklist for your platform. It is one page and it tells you what "ready" means.
2. Chapters 05 and 06, which are the two that prevent the most future pain.
3. Chapter 27, before you touch a single API key.
4. Everything else on the day you need it.

LAB 1 – BEFORE ANYTHING ELSE

1. Write down, on paper, your operating system and its version, your processor (Apple Silicon, Intel, AMD), your RAM, your free disk space, and whether you have an NVIDIA GPU.
2. Write down whether you can run an installer without asking someone for a password.
3. Keep that paper inside this manual. Chapter 02 tells you what it means, and every troubleshooting conversation for the next year will start with it.

PART I — GROUND

02 Your machine: hardware, OS, and what it can actually run

Before you install anything, you need an honest picture of the machine in front of you. Most of the frustration in a first month comes from trying to run work the hardware cannot do, and then blaming the software. This chapter turns your machine into three facts: what you can run locally, what you must send elsewhere, and what will need an alternative install path.

Find out what you have

```
$ # macOS
sysctl -n machdep.cpu.brand_string; uname -m
system_profiler SPHardwareDataType | grep -E "Memory|Chip"
df -h /

$ # Linux / WSL2
lscpu | head -20; free -h; df -h /
nvidia-smi          # NVIDIA GPU present? See chapter 22

PS> # Windows PowerShell
Get-ComputerInfo -Property CsName,OsName,CsTotalPhysicalMemory,CsSystemType
Get-CimInstance Win32_VideoController | Select-Object Name,AdapterRAM
```

Listing 2.1 — Machine inventory. `uname -m` returning `arm64` means Apple Silicon; `x86_64` means Intel or AMD.

What each class of machine can do

MACHINE	RUNS LOCALLY WITHOUT COMPLAINT	SEND ELSEWHERE
Apple Silicon, 16 GB+	API-based work, small local models via Ollama or MLX, embeddings, all of Parts I–III	Training, anything requiring CUDA, large model fine-tuning
Apple Silicon, 8 GB	API-based work, 3B-class quantised models, notebooks	Anything above ~7B parameters; expect swapping
Windows/Linux + NVIDIA GPU, 8 GB+ VRAM	Local inference, small fine-tunes, CUDA work, GPU containers	Multi-GPU training, 70B-class models
No GPU at all, any OS	Everything in this manual except chapter 22's local GPU labs	All model training and local large-model inference
Locked-down work laptop	User-level Python, git, VS Code, API work	Docker Desktop, WSL2 install, anything needing a driver
Nothing local (Colab / rented)	All of it, in a browser, with the caveats in chapter 21	Nothing — but read chapter 23 on cost first

GOLDEN RULE 02

A GPU is not a prerequisite for AI engineering. It is a prerequisite for training. Most of the work — and most of the employment — is on the other side of an API.

Disk space is the constraint nobody plans for

Model weights, container images and package caches are large and invisible. A single 7B model in half precision is about 14 GB. A CUDA-enabled PyTorch install is 3–5 GB. Docker images for machine learning routinely exceed 10 GB. Plan on 100 GB free before you start, and read chapter 26 the first time your disk fills, which it will.

Two paths: admin and no admin

Wherever installs differ, this manual gives both. The pattern is consistent: the admin path uses a system package manager, the no-admin path installs into your home directory and adds a line to your shell profile.

TOOL	WITH ADMIN RIGHTS	WITHOUT ADMIN RIGHTS
Python 3.11	Homebrew, apt, or the python.org installer	<code>uv python install 3.11</code> , or miniforge into <code>~/miniforge3</code>
Git	<code>brew install git</code> / <code>apt install git</code>	Usually already present; otherwise conda-forge, or GitHub Desktop
VS Code	System installer	User installer (Windows) or the portable ZIP
Docker	Docker Desktop	Not possible — use Colab or a rented machine (chapters 21, 23)
WSL2	<code>wsl --install</code>	Not possible — it needs a Windows feature toggle

Which platform should you actually work on?

If you have a Mac, work on macOS directly. If you have Windows and administrator rights, install WSL2 and do all your work inside it: almost every AI tutorial, Dockerfile and shell script in existence assumes Linux, and WSL2 gives you Linux without giving up Windows. If you have Windows without administrator rights, work in native PowerShell and accept that a minority of instructions will need translating — this manual translates them for you. If your machine is old, slow or locked down beyond use, work in Colab and treat the browser as your machine.

Windows users: files inside WSL2 (`/home/you/...`) are fast; files on the Windows side reached from WSL2 (`/mnt/c/...`) are slow, sometimes ten times slower. Keep your projects inside the Linux filesystem and open them with `code .` from a WSL2 shell.

When it goes wrong

SYMPTOM	CAUSE	FIX
Everything is extremely slow, fan constantly on	Out of RAM; the OS is swapping to disk.	Close browsers, use a smaller model, or move the job to Colab. Check with <code>top</code> / Task Manager.
Illegal instruction or Bad CPU type on a Mac	An x86 binary on Apple Silicon, or a Rosetta-mixed environment.	Rebuild the environment natively; check <code>python -c "import platform; print(platform.machine())"</code> says <code>arm64</code> .
<code>nvidia-smi</code> : command not found with an NVIDIA card installed	Driver not installed, or you are inside WSL2 without the Windows driver.	Chapter 22. On WSL2 the driver is installed on Windows, never inside Linux.
Installer refuses: "you need administrator privileges"	Managed machine.	Use the no-admin column above. Do not fight the policy.
Disk full after a week	Model and package caches.	Chapter 26.

LAB 2 — KNOW YOUR MACHINE

1. Run the inventory commands for your platform from Listing 2.1.
2. Find your row in the capability table and write the two lists — local, elsewhere — on the same paper as Lab 1.
3. Note whether you have admin rights, and which column of the install table you will be using for the rest of this manual.

Verify — you can state your architecture, your RAM, your free disk space and your GPU situation without looking anything up.

Where to look it up

developer.apple.com/metal/pytorch — Apple Silicon acceleration

learn.microsoft.com/windows/wsl — WSL2 install and GPU support

docs.nvidia.com/cuda/wsl-user-guide — NVIDIA on WSL2

PART I — GROUND

03 The terminal: shells, commands, and the twenty you need

The terminal is a program that reads a line of text, runs it, and prints what happened. That is all it is. The intimidation comes from the blank prompt, which tells you nothing about what it accepts — so this chapter gives you the twenty commands that cover the great majority of daily work, and the four ideas that let you combine them.

Which shell am I in?

A shell is the program interpreting your typing. macOS defaults to **zsh**; most Linux systems and WSL2 default to **bash**. They are close enough that nearly every command in this manual works in both, and where they differ this manual says so. Windows PowerShell is genuinely different — a different language with different command names — and gets its own lines throughout.

```
$ echo $SHELL          # /bin/zsh or /bin/bash
$ echo $0              # the shell running right now
PS> $PSVersionTable    # PowerShell version
```

The twenty

MACOS / LINUX / WSL2	POWERSHELL	WHAT IT DOES
<code>pwd</code>	<code>pwd</code>	Where am I?
<code>ls -la</code>	<code>ls -Force</code>	List everything here, hidden files included
<code>cd path</code>	<code>cd path</code>	Go there. <code>cd ..</code> up one, <code>cd ~</code> home, <code>cd -</code> back
<code>mkdir -p a/b</code>	<code>mkdir a\b</code>	Make a directory, parents included
<code>cp a b / cp -r</code>	<code>Copy-Item a b -Recurse</code>	Copy a file or a tree
<code>mv a b</code>	<code>Move-Item a b</code>	Move or rename
<code>rm f / rm -rf d</code>	<code>Remove-Item d -Recurse</code>	Delete. No undo. Ever.
<code>cat f</code>	<code>Get-Content f</code>	Print a whole file
<code>less f</code>	<code>more f</code>	Page through a file; <code>q</code> to quit
<code>head -20 f / tail -f log</code>	<code>Get-Content f -Head 20 / -Wait</code>	First lines; follow a growing log
<code>grep -rn "text" .</code>	<code>Select-String -Path * -Pattern "text"</code>	Find text inside files
<code>find . -name "*.py"</code>	<code>Get-ChildItem -Recurse -Filter *.py</code>	Find files by name
<code>which python</code>	<code>where.exe python</code>	Which program will actually run
<code>curl -O url</code>	<code>curl.exe -O url</code>	Download a file
<code>du -sh *</code>	<code>Get-ChildItem measure Length -Sum</code>	What is taking up space
<code>top / htop</code>	Task Manager	What is using the CPU
<code>kill -9 PID</code>	<code>Stop-Process -Id PID</code>	End a stuck process
<code>chmod +x f</code>	n/a	Make a file executable
<code>export VAR=x</code>	<code>\$env:VAR = "x"</code>	Set an environment variable for this session
<code>history</code>	<code>Get-History</code>	What did I just run?

The four ideas

Ctrl-C stops the current program. Not Ctrl-Z, which suspends it and leaves it lurking. If Ctrl-C does nothing, close the window; if a background job is stuck, find it with `top` and end it by PID.

Tab completes. Type three characters of a filename and press Tab. This is not a convenience — it is how you avoid typos in paths, which are the single most common cause of "command not found".

Up-arrow and Ctrl-R recall. Up-arrow walks back through recent commands; Ctrl-R searches them. You will retype almost nothing after a week.

The pipe passes output along. `|` feeds one command's output into the next, `>` writes it to a file, `>>` appends. `pip list | grep torch` is the whole idea in one line.

GOLDEN RULE 03

Read the last line of an error first. Then the first. The middle is usually a stack trace of code you did not write.

Never paste a command you do not understand into a terminal, especially one that begins `curl ...` or `sudo bash`. Read it, or ask what it does, before you run it. This is not paranoia: it is the one irreversible mistake available to a beginner.

When it goes wrong

SYMPTOM	CAUSE	FIX
command not found	Typo, or the program is not on your PATH.	Tab-complete it. Then chapter 04 on PATH.
No such file or directory for a file you can see	You are in a different directory than you think, or the path has a space in it.	<code>pwd</code> , then <code>ls</code> . Quote paths with spaces: <code>cd "My Folder"</code> .
Permission denied	Not executable, or not yours.	<code>chmod +x file</code> ; check ownership with <code>ls -l</code> . Chapter 04.
Terminal frozen after pressing something	Ctrl-S froze output, or a program is waiting for input.	Ctrl-Q to resume, Ctrl-C to abort, Ctrl-D to signal end-of-input.
Copy-paste puts weird characters in your command	Smart quotes from a document or web page.	Retype the quotes by hand. This eats an hour of somebody's week, every week.

LAB 3 — TWENTY MINUTES AT THE PROMPT

1. From your home directory, create `~/projects/summarise-cli` in one command.
2. `cd` into it, create an empty `notes.md`, write one line into it with `echo` and `>`, then print it.
3. List the ten largest things in your home directory using `du` and a pipe.
4. Find every `.py` file in your home directory that contains the word `import`.

Verify — you did all four without using a mouse or a file browser.

04 Files, paths, permissions and PATH itself

Two different things are called "path" and confusing them causes a specific, recurring bewilderment. A *file path* is where something lives. `PATH`, in capitals, is a list of directories your shell searches when you type a command name. Almost every "I installed it but the computer says it does not exist" problem is a `PATH` problem.

Absolute, relative, home

An absolute path starts at the root and is unambiguous: `/Users/you/projects/x`, `C:\Users\you\projects\x`. A relative path starts where you are: `./x` here, `../x` one level up. `~` means your home directory. Use absolute paths in configuration files and cron jobs; use relative paths inside a project so it still works on someone else's machine.

Windows separates directories with `\`, everything else with `/`. In Python always use `pathlib`: `Path("data") / "input.txt"` produces the right separator everywhere, and this single habit removes a whole class of cross-platform bug.

Hidden files, and why your project is full of them

A leading dot hides a file from ordinary listings: `.venv`, `.env`, `.git`, `.gitignore`. They are not secret, only quiet. `ls -la` shows them, and you should get in the habit of using it, because the files that matter most in this manual are all dotfiles.

Permissions in one page

`ls -l` prints something like `-rw-r--r-- 1 you staff 2048 May 17 09:12 notes.md`. The ten characters are: type, then three groups of read/write/execute for the owner, the group, and everyone else. `chmod +x script.sh` adds execute permission so you can run it. `chmod 600 ~/.env` means only you can read or write it — do this to every file containing a secret (chapter 27).

```
$ ls -l script.sh
-rw-r--r-- 1 you staff 412 May 17 09:12 script.sh
$ chmod +x script.sh && ./script.sh
$ chmod 600 .env && ls -l .env
-rw----- 1 you staff 128 May 17 09:14 .env
```

Listing 4.1 — Making a script runnable, and locking down a secrets file.

PATH: the list that decides what runs

When you type `python`, the shell walks `PATH` from left to right and runs the first match it finds. *Left to right* is the whole game: this is why activating a virtual environment — which prepends a directory — changes which Python you get, and why a stale entry in your profile can silently shadow a fresh install.

```
$ echo $PATH | tr ':' '\n'      # one directory per line, in search order
$ which -a python python3    # every match, not just the winner
PS> $env:PATH -split ';'
PS> where.exe python
```

Listing 4.2 — Reading PATH. `which -a` is the single most useful diagnostic command in this manual.

Where PATH comes from, per platform

PLATFORM	FILE TO EDIT	LINE TO ADD
macOS (zsh, default)	<code>~/.zshrc</code>	<code>export PATH="\$HOME/.local/bin:\$PATH"</code>
Linux / WSL2 (bash)	<code>~/.bashrc</code>	<code>export PATH="\$HOME/.local/bin:\$PATH"</code>
Login shells (either)	<code>~/.profile</code>	Same line; used when <code>.bashrc</code> is not read
Windows PowerShell	<code>\$PROFILE</code>	<code>\$env:PATH = "\$HOME\local\bin;" + \$env:PATH</code>
Windows, permanently	System settings → Environment Variables	Edit the user <code>Path</code> entry; no admin needed

After editing, either open a new terminal or run `source ~/.zshrc` (`. $PROFILE` in PowerShell). Editing the file does nothing to the session you are already in — this trips up everyone once.

GOLDEN RULE 04

When a tool behaves impossibly, run `which -a <tool>`. You are almost certainly running a different copy than the one you think you installed.

When it goes wrong

SYMPTOM	CAUSE	FIX
Installed a tool, shell still says <code>command not found</code>	Its directory is not on PATH, or the shell has not reloaded.	Add the line from the table, open a new terminal, verify with <code>which -a</code> .
Two versions of a tool, wrong one wins	An earlier PATH entry shadows the later one.	<code>which -a tool</code> , then reorder PATH. Do not delete the loser; put the winner first.
Works in the terminal, fails in VS Code or cron	Those start a different kind of shell and read a different profile file.	Put the export in <code>~/.profile</code> as well; in VS Code, select the interpreter explicitly (chapter 14).
<code>Permission denied</code> running your own script	Missing execute bit.	<code>chmod +x script.sh</code>
PATH looks doubled or enormous	An export line lives in two profile files and runs twice.	Grep your dotfiles: <code>grep -n "PATH" ~/.zshrc ~/.bashrc ~/.profile</code> . Keep one.
<code>~/.env</code> readable by everyone	Default file permissions.	<code>chmod 600 ~/.env</code> . Chapter 27.

LAB 4 – SUMMARISE-CLI, STEP 1

1. Print your PATH one directory per line. Count them. Find the entry that supplies your current `python3`.
2. Create `~/.local/bin` and add it to the front of your PATH in the right file for your shell.
3. Write a two-line shell script there called `hello-ai` that prints your name, make it executable, and run it from a different directory by name alone.
4. Inside `~/projects/summarise-cli`, create `data/` and put a plain text file of a few paragraphs in it. That file is the input your tool will summarise for the rest of this manual.

Verify — `which hello-ai` prints a path in `~/.local/bin`, and the command works from anywhere.

PART I – WHAT AGES FASTEST

Hardware advice. The capability table in chapter 02 assumes 2026-era laptops and model sizes; re-read it against whatever "small model" means when you next open this book. The shell, PATH and permissions material in chapters 03 and 04 has been true for forty years and will outlive both of us.

05 Python itself: interpreters, versions, and which one you are running

Your machine does not have "Python" on it. It has several Pythons, installed by different things for different reasons, and they do not know about each other. The system one that came with the OS. One from Homebrew or apt. One from an installer you ran last year. One inside every virtual environment. One inside each conda environment. Knowing which one you are talking to at any moment is the foundational skill of this entire part.

Find every Python on your machine

```
$ which -a python python3 python3.11
$ python3 -V
$ python3 -c "import sys; print(sys.executable)"
PS> where.exe python; py --list
```

Listing 5.1 — `sys.executable` is the truth: it is the interpreter actually running your code, whatever the shell claims.

On macOS, `python` with no version number may not exist at all, and the `python3` at `/usr/bin/python3` belongs to Apple. Never install packages into it and never delete it — parts of the OS use it. Install your own.

Why 3.11, and why pinned

This manual pins Python 3.11 everywhere. The reason is not that 3.11 is best; it is that the machine learning ecosystem lags the language. Compiled packages — PyTorch, numpy, scipy, tokenizers — ship pre-built wheels for a specific set of versions, and being one version ahead of the ecosystem means waiting for a compiler to fail on your laptop instead of downloading a working binary in ten seconds. Being one or two versions behind the newest release is the sweet spot for this kind of work, permanently. When 3.11 is genuinely old, apply the same reasoning again rather than the same number.

GOLDEN RULE 05

Never install project packages into the Python that came with your operating system. Not once, not "just to test". It is the fastest route to a broken machine.

Installing 3.11, both paths, every platform

PLATFORM	WITH ADMIN	WITHOUT ADMIN
macOS	<code>brew install python@3.11</code>	<code>uv python install 3.11</code> , or miniforge (chapter 08)
Ubuntu / Debian / WSL2	<code>sudo apt install python3.11</code> <code>python3.11-venv</code>	<code>uv python install 3.11</code>
Windows	python.org installer, tick "Add to PATH"	Same installer, "Install for me only"; or <code>uv python install 3.11</code>
Any, version-juggling	<code>pyenv install 3.11</code>	<code>uv</code> — no compiler needed, downloads a prebuilt interpreter

On Ubuntu, `python3.11-venv` is a separate package and its absence produces a confusing failure when you try to make an environment. Install it at the same time.

Multiple versions side by side

Call the version explicitly and the ambiguity disappears: `python3.11 -m venv .venv` creates an environment from a known interpreter, no matter what `python` happens to mean today. On Windows, the `py` launcher does the same job: `py -3.11 -m venv .venv` . Record the version your project needs in a `.python-version` file at the project root; `pyenv` and `uv` both read it, and humans read it too.

When it goes wrong

SYMPTOM	CAUSE	FIX
<code>python</code> : command not found but <code>python3</code> works	No unversioned alias exists; correct and normal on modern systems.	Type <code>python3</code> , or work inside a virtual environment where <code>python</code> exists.
Windows opens the Microsoft Store when you type <code>python</code>	Windows' app-execution alias stub.	Settings → App execution aliases → turn off the Python stubs. Or use <code>py</code> .
No module named <code>venv</code>	Debian/Ubuntu split it into its own package.	<code>sudo apt install python3.11-venv</code>
Installing a package tries to compile and fails with a wall of C errors	No prebuilt wheel for your Python version or architecture.	Use a Python version the package publishes wheels for — usually one or two behind the newest. Chapter 12.
System tools break after installing packages with <code>sudo pip</code>	You overwrote OS-managed libraries.	Golden rule 05. Recovery is per-distro and unpleasant; a fresh user account is sometimes faster.
<code>brew</code> upgraded Python and every environment broke	Environments link to the base interpreter, which moved.	Rebuild environments from the lockfile. Chapter 13.

LAB 5 – CENSUS OF INTERPRETERS

1. List every Python on your machine with `which -a`, and for each one print its version and `sys.executable`.
2. Identify which is the OS's, which is yours, and which one bare `python3` currently resolves to.
3. Install 3.11 by the path that suits your rights, and create `~/projects/summarise-cli/.python-version` containing `3.11`.

Verify — `python3.11 -V` prints a 3.11.x version, and you can say where that binary lives.

Where to look it up

docs.python.org/3/using – installing and invoking Python per platform

devguide.python.org/versions – which versions are still supported

06 Virtual environments: the mental model

Almost every environment problem you will hit for the rest of your career is one question asked badly: *which Python am I talking to, and where is it putting things?* A virtual environment is the answer made physical. It is a directory. Inside it sits a Python interpreter (or a link to one) and a `site-packages` folder, and nothing else on your machine knows or cares about it.

Activating an environment does one modest thing: it puts that directory's `bin` (or `Scripts` on Windows) at the front of your `PATH`. That is the whole trick. Understand this one sentence and half the troubleshooting tables in this manual become obvious.

What is actually inside

```
.venv/
├─ bin/           # Scripts/ on Windows
│   ├─ python     # link to the base interpreter
│   └─ pip
├─ activate       # the script that edits your PATH
├─ lib/python3.11/site-packages/ # everything you install
└─ pyvenv.cfg     # which base Python this was built from
```

Figure 6.1 — An environment is a folder, not a setting. Deleting the folder deletes the environment completely and harmlessly.

Create, activate, verify, leave

```
$ # macOS / Linux / WSL2 – zsh and bash are identical here
python3.11 -m venv .venv
source .venv/bin/activate
which python          # must end in .venv/bin/python
deactivate

PS> # Windows PowerShell
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
where.exe python
deactivate
```

Listing 6.1 — The four operations. There are no others.

GOLDEN RULE 06

One project, one environment, one Python version, written down in the repository. Name it `.venv`, keep it inside the project, and never commit it.

Conventions worth adopting on day one

- **Name it `.venv`.** VS Code finds it automatically, `uv` and `poetry` expect it, and every `.gitignore` template already excludes it.
- **Keep it inside the project.** One folder to delete when things go wrong; no doubt about which environment belongs to which code.
- **Never commit it.** It contains machine-specific binaries and thousands of files. Commit the lockfile instead (chapter 13).
- **Recreating is normal.** An environment is disposable. If it misbehaves for more than ten minutes, delete it and rebuild — that is a thirty-second operation once chapter 13 is in place, not a defeat.

You do not always need to activate

Activation is a convenience for interactive work. Two habits avoid it entirely and are more reliable in scripts, editors and CI: call the interpreter by its full path (`.venv/bin/python script.py`), or use a runner that resolves the environment itself (`uv run script.py`). Both are immune to the most common confusion in this chapter, which is an activated environment in one terminal tab and not another.

When it goes wrong

SYMPTOM	CAUSE	FIX
<code>ModuleNotFoundError</code> for a package you just installed	The install went to a different interpreter than the one running your script.	Always <code>python -m pip install</code> , never bare <code>pip</code> . Compare <code>which python</code> and <code>which pip</code> .
Prompt shows <code>(.venv)</code> but imports still fail	Your editor or notebook is using a different interpreter than your terminal.	VS Code: <i>Python: Select Interpreter</i> → the one in <code>./.venv</code> . Chapters 14 and 20.
<code>Activate.ps1</code> cannot be loaded because running scripts is disabled	PowerShell execution policy.	<code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code> — no admin needed.
<code>source: no such file or directory: .venv/bin/activate</code>	Creation failed, or you are on Windows where the path is <code>Scripts</code> .	Check the folder exists; on Ubuntu install <code>python3.11-venv</code> and recreate.
Environment broke after an OS or Homebrew update	The base interpreter it links to was replaced.	Delete <code>.venv</code> and rebuild from the lockfile. This is why chapter 13 exists.
Moved or renamed the project folder and nothing works	Paths inside the environment are absolute.	Delete and recreate. Never move an environment.
Two environments active at once, prompt shows both	Activated one inside another.	<code>deactivate</code> until the prompt is clean, then activate one.

LAB 6 — SUMMARISE-CLI, STEP 2

1. Give `summarise-cli` a 3.11 environment named `.venv`.
2. Install nothing yet. Run `python -m pip list` and read the two or three lines it prints.
3. Deactivate, run the same command, and compare. Write the difference down in your own words.
4. Delete `.venv` entirely and recreate it, to prove to yourself that it costs nothing.

Verify — `python -c "import sys; print(sys.prefix)"` prints a path inside your project.

07

PART II — PYTHON

pip

pip installs packages from the Python Package Index into whichever interpreter runs it. It is the lowest common denominator: present everywhere, understood by everything, and the tool every other tool in this part is ultimately wrapping. Learn it first even if you end up using something else daily, because when a fancier tool fails, the error you have to read is usually pip's.

The commands that matter

COMMAND	WHAT IT DOES
<code>python -m pip install requests</code>	Install the newest compatible version
<code>python -m pip install "openai==1.40.0"</code>	Install exactly that version
<code>python -m pip install -r requirements.txt</code>	Install everything a project declares
<code>python -m pip install -e .</code>	Install this project itself, editable — your code changes take effect without reinstalling
<code>python -m pip uninstall requests</code>	Remove it, but not the things it dragged in
<code>python -m pip list</code> / <code>list --outdated</code>	What is installed; what has newer releases
<code>python -m pip show -f numpy</code>	Version, location, dependencies, files
<code>python -m pip freeze > requirements.txt</code>	Write down exactly what is installed right now
<code>python -m pip check</code>	Report installed packages with broken dependencies
<code>python -m pip cache dir</code> / <code>purge</code>	Where downloads are cached; empty it
<code>python -m pip download pkg -d ./wheels</code>	Fetch without installing — for offline or air-gapped machines
<code>python -m pip install --no-deps pkg</code>	Install only this, resolving nothing. A precision tool; usually a mistake.

GOLDEN RULE 07

Write `python -m pip`, never bare `pip`. The long form installs into the Python you are running; the short form installs into whichever pip won the PATH race.

requirements.txt, and what freeze gets wrong

`pip freeze` captures everything installed, including packages that arrived only as dependencies of something else, with no record of which. Six months later nobody can tell what the project actually asked for. The discipline that fixes this costs nothing: keep a hand-written `requirements.in` listing only your direct dependencies, and generate the full pinned `requirements.txt` from it. Chapter 13 shows the whole pattern.

```
$ # direct dependencies, hand-written, with intent
cat requirements.in
openai>=1.40
python-dotenv

$ # the exact world, generated — commit both files
python -m pip install pip-tools
pip-compile requirements.in -o requirements.txt
python -m pip install -r requirements.txt
```

Listing 7.1 — Two files, two purposes: what you want, and what you got.

Version specifiers, read correctly

SPECIFIER	MEANS
<code>==1.40.0</code>	Exactly this. Use in lockfiles.
<code>>=1.40</code>	This or newer. Fine for libraries, risky unpinned in applications.
<code>~1.40.0</code>	1.40.x — patch updates only.
<code>>=1.40,<2</code>	The usual sane range: bug fixes and features, no breaking changes.
<code>!=1.41.2</code>	Anything but that one broken release.

Where packages come from

By default, PyPI. Behind a corporate proxy or an internal mirror you will need `--index-url` or a `pip.conf`; PyTorch's CUDA builds live on their own index and need `--index-url` too (chapter 22). Configuration lives at `~/.config/pip/pip.conf` on macOS and Linux, `%APPDATA%\pip\pip.ini` on Windows. Appendix C covers proxies in full.

When it goes wrong

SYMPTOM	CAUSE	FIX
<code>error: externally-managed-environment</code>	You are installing into a system Python that the OS protects. Good.	Make and activate a virtual environment. Never reach for <code>--break-system-packages</code> .
<code>Defaulting to user installation because normal site-packages is not writeable</code>	No environment active; pip is installing into <code>~/.local</code> .	Activate an environment. This message is a warning that your install will not be where you expect.
<code>Could not build wheels for X</code> , pages of compiler output	No prebuilt wheel for your Python or platform, and no compiler.	Check the package's supported versions; drop to a Python it ships wheels for. Chapter 12.
<code>ResolutionImpossible</code>	Two requirements genuinely conflict.	Read the conflict lines from the bottom up. Chapter 12 works a real example.
<code>SSL: CERTIFICATE_VERIFY_FAILED</code>	Corporate TLS inspection.	Appendix C. Add the corporate root certificate; do not use <code>--trusted-host</code> permanently.
Install succeeds, import fails	Installed into a different interpreter.	<code>python -m pip show pkg</code> and compare its Location with <code>sys.path</code> .
Everything reinstalls from scratch every time	Cache disabled or purged, or a container without a mounted cache.	<code>pip cache dir</code> ; in Docker, mount it (chapter 25).
An install silently broke another project	You installed outside an environment, into the user site.	<code>python -m pip list --user</code> to see the damage; move to per-project environments.

LAB 7 – SUMMARISE-CLI, STEP 3

1. In the project environment, write `requirements.in` with two lines: `openai>=1.40` and `python-dotenv`.
2. Compile it to a pinned `requirements.txt` and install from that file.
3. Run `python -m pip list` and count how many packages arrived that you did not name. Use `pip show` to find out who asked for two of them.
4. Write `summarise.py` that reads your text file and prints its first 200 characters. No API call yet — just prove the file loads.

Verify — `python -m pip check` reports no broken dependencies, and `python summarise.py data/input.txt` prints text.

Where to look it up

pip.pypa.io – every flag, current behaviour

packaging.python.org/tutorials/installing-packages – the official mental model

08

PART II — PYTHON

conda and miniforge

conda solves a problem pip does not: it installs non-Python things. Compilers, CUDA runtimes, system libraries like FFmpeg and GDAL, and the Python interpreter itself are all just packages to conda. In scientific and GPU work that matters, because the hardest installs are usually not Python at all.

Which conda to install

Install **miniforge**. It is minimal, it defaults to the community conda-forge channel, and it avoids the licensing questions attached to Anaconda's own distribution for commercial use. Avoid full Anaconda — it installs several gigabytes you did not ask for. Miniconda is acceptable but points at the commercial default channel.

```
$ # macOS / Linux / WSL2 — installs into your home directory, no admin
curl -L -O "https://github.com/conda-
forge/miniforge/releases/latest/download/Miniforge3-$(uname)-$(uname -m).sh"
bash Miniforge3-$(uname)-$(uname -m).sh -b -p "$HOME/miniforge3"
"$HOME/miniforge3/bin/conda" init zsh    # or bash

$ # keep base clean: never install project packages into it
conda config --set auto_activate_base false
```

Listing 8.1 — Miniforge, user-level. On Windows use the `.exe` installer and choose "Just Me".

Daily commands

COMMAND	WHAT IT DOES
<code>conda create -n aie python=3.11</code>	New environment with a named Python
<code>conda activate aie</code> / <code>conda deactivate</code>	Enter and leave
<code>conda install -c conda-forge numpy ffmpeg</code>	Install from a channel, Python and non-Python alike
<code>conda env list</code>	Every environment and where it lives
<code>conda list</code>	What is in this one, and which channel it came from
<code>conda env export --from-history > environment.yml</code>	Export what you <i>asked for</i> — portable across platforms
<code>conda env create -f environment.yml</code>	Rebuild from that file
<code>conda env remove -n aie</code>	Delete an environment
<code>conda clean --all</code>	Reclaim disk space from the package cache
<code>conda info</code>	Channels, paths, config — the first thing to paste when asking for help

GOLDEN RULE 08

Do not mix conda and pip carelessly in one environment. If you must, install everything you can with conda first, pip last, and never let pip overwrite a conda-installed package.

Mixing with pip, when you have to

Some packages exist only on PyPI. The safe pattern is to declare both in one file so the environment is still reproducible, and to install conda's half first — conda has no visibility into what pip does afterwards, so pip going last is the only order where conda's solver is not working from a stale picture.

```
name: aie
channels: [conda-forge]
dependencies:
  - python=3.11
  - numpy
  - jupyterlab
  - pip
  - pip:
    - openai>=1.40
    - python-dotenv
```

Listing 8.2 — `environment.yml` with a pip section. Recreate with `conda env create -f environment.yml`.

Why it is slow, and what to do

conda's dependency solving is a genuinely hard computation, and on a large environment it can take minutes. Modern conda uses the fast libmamba solver by default; if yours does not, `conda install -n base conda-libmamba-solver` and `conda config --set solver libmamba`. The other half of the answer is to keep environments small and single-purpose rather than accumulating one enormous environment for everything.

When it goes wrong

SYMPTOM	CAUSE	FIX
<code>conda</code> : command not found after installing	<code>conda init</code> not run, or shell not restarted.	<code>~/miniforge3/bin/conda init <shell></code> , then open a new terminal.
Solving environment: stuck for many minutes	Classic solver, too many loose constraints, or too many channels.	Switch to libmamba; pin Python; use only conda-forge.
<code>PackagesNotFoundError</code>	Not in your channels, or not built for your platform (common on Apple Silicon).	Add <code>-c conda-forge</code> ; check the package's platform support; fall back to pip.
Environment worked, then broke after a pip install	pip overwrote a conda-managed package.	Rebuild from <code>environment.yml</code> . Follow Golden rule 08.
<code>(base)</code> always in your prompt	Auto-activation.	<code>conda config --set auto_activate_base false</code>
<code>environment.yml</code> from a colleague will not build	It was exported with full builds, which are platform-specific.	Ask for <code>--from-history</code> , or strip the build strings.
Tens of gigabytes gone	The package cache, which is never cleaned automatically.	<code>conda clean --all</code> . Chapter 26.

LAB 8 – THE SAME PROJECT, IN CONDA

1. Create a conda environment `aie` with Python 3.11, and install `jupyterlab` from conda-forge.
2. Export it with `--from-history` and read the file. Compare its length to `conda env export` without the flag.
3. Run your `summarise.py` from chapter 07 inside this environment. Note what you had to install again, and why.
4. Keep the conda environment for notebook work; keep `.venv` for the tool. Both can coexist.

Verify — `conda env list` shows `aie`, and `conda list | wc -l` is under a hundred.

Where to look it up

conda-forge.org/docs – channel policy and platform support

github.com/conda-forge/miniforge – installers

09

PART II – PYTHON

uv

uv is a single binary that replaces most of what pip, venv, pip-tools and pyenv do, and it is fast enough that the difference changes how you work — installs that took a minute take a second or two, so recreating an environment stops being a decision and becomes a reflex. It also installs Python interpreters themselves, which makes it the best answer in this manual for anyone without administrator rights.

Install

```
$ # macOS / Linux / WSL2 – into ~/.local/bin, no admin
curl -LsSf https://astral.sh/uv/install.sh | sh

PS> # Windows, user-level
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"

$ # or, if you already have any Python
python -m pip install --user uv
```

Listing 9.1 — Read the script before piping it to a shell if you can; chapter 03 said so and meant it.

Two ways to use it

As a faster pip. Every pip command you know works with `uv pip` in front, against an ordinary `.venv`. This is the zero-commitment way in, and it interoperates with everything.

```
$ uv venv --python 3.11          # creates .venv, downloads 3.11 if missing
$ source .venv/bin/activate
$ uv pip install openai python-dotenv
$ uv pip compile requirements.in -o requirements.txt
$ uv pip sync requirements.txt   # make the env match the file exactly
```

As a project manager. `uv init` creates a `pyproject.toml`; `uv add` records a dependency, resolves it, and writes a `uv.lock`; `uv run` executes a command in the project environment, creating or updating it first if needed. There is no activation step at all.


```
$ uv init summarise-cli && cd summarise-cli
$ uv add openai python-dotenv
$ uv add --dev pytest ruff
$ uv run python summarise.py data/input.txt
$ uv sync                                # exact environment from uv.lock
$ uv lock --upgrade-package openai
$ uv tool install ruff                   # global CLI tools, isolated
$ uv python install 3.11 3.12           # interpreters, no admin, no compiler
```

Listing 9.2 — `uv.lock` is committed; `.venv` never is.

GOLDEN RULE 09

Commit the lockfile, whichever tool produces it. An environment nobody can rebuild is a liability disguised as a working machine.

`uv` is young compared with `pip` and `conda`. That means it moves fast and its flags change; check the official documentation rather than a two-year-old blog post, and expect the commands above to grow rather than break.

When it goes wrong

SYMPTOM	CAUSE	FIX
<code>uv: command not found</code> after install	<code>~/local/bin</code> not on PATH.	Chapter 04's table, then a new terminal.
<code>uv run</code> uses a different Python than expected	A <code>.python-version</code> file, or <code>requires-python</code> in <code>pyproject.toml</code> .	<code>uv run python -V</code> , then reconcile the two declarations.
Lockfile conflicts constantly in git	Several people resolving independently.	One person runs <code>uv lock</code> ; everyone else runs <code>uv sync</code> . Chapter 16.
A package installs with pip but not uv	Unusual source, legacy metadata, or a private index.	Configure the index with <code>--index-url</code> ; fall back to <code>uv pip install</code> for that one package.
CI is slow despite uv	The cache is not persisted between runs.	Cache <code>~/cache/uv</code> . Chapter 29.

LAB 9 – SUMMARISE-CLI, STEP 4

1. Install uv. Time `uv venv` against `python -m venv` on the same folder.
2. Convert `summarise-cli` to a uv project: `uv init`, then `uv add` your two dependencies. Read the generated `pyproject.toml`.
3. Delete `.venv`, run `uv sync`, and time how long a full rebuild takes.
4. Run your script with `uv run` — no activation.

Verify — `uv.lock` exists, is committed, and a full rebuild from it takes seconds.

Where to look it up

docs.astral.sh/uv – the reference; assume it is newer than this page

10

PART II – PYTHON

poetry

poetry treats your work as a package rather than a folder of scripts. It manages dependencies, separates production from development requirements, resolves and locks them, and builds and publishes distributions. If you are writing a library other people will install, this is the discipline you want. If you are writing a training script, it is more ceremony than the job needs.

Install and use

```
$ curl -sSL https://install.python-poetry.org | python3 - # user-level
$ poetry config virtualenvs.in-project true                # keep .venv in the
project

$ poetry new summarise-cli # or `poetry init` in an existing folder
$ poetry add openai python-dotenv
$ poetry add --group dev pytest ruff
$ poetry install          # from poetry.lock
$ poetry run python summarise.py data/input.txt
$ poetry lock              # re-resolve after editing pyproject.toml
$ poetry show --tree       # why is this package here?
$ poetry build             # wheel + sdist in dist/
```

Listing 10.1 — `poetry show --tree` is the best dependency explainer of any tool in this part.

What pyproject.toml is

One file describing the project: its name, version, Python requirement, dependencies, dependency groups, and build configuration. It is a standard, not a poetry invention — uv, pip and hatch all read it — which is why learning it pays off regardless of which tool you settle on.

```
[project]
name = "summarise-cli"
version = "0.1.0"
requires-python = ">=3.11,<3.13"
dependencies = ["openai>=1.40", "python-dotenv>=1.0"]

[project.scripts]
summarise = "summarise_cli.main:main" # installs a real command

[dependency-groups]
dev = ["pytest>=8", "ruff>=0.6"]
```

Listing 10.2 — The `[project.scripts]` entry is how `summarise-cli` becomes a command you can type, rather than a file you run.

When it goes wrong

SYMPTOM	CAUSE	FIX
Locking takes many minutes	Wide version ranges force the resolver to explore an enormous space.	Constrain <code>requires-python</code> and upper-bound the big packages.
<code>SolverProblemError</code>	Genuine conflict, often a too-tight <code>python</code> constraint.	Read which two packages disagree; relax one bound. Chapter 12.
Environment created somewhere surprising	Default is a central cache directory, not the project.	<code>poetry config virtualenvs.in-project true</code> , then recreate.
<code>poetry: command not found</code>	Installer put it in <code>~/.local/bin</code> .	Chapter 04.
Works locally, fails in Docker or CI	<code>poetry.lock</code> not committed, or dev group installed in production.	Commit the lock; <code>poetry install --without dev</code> in images.
Torch or a CUDA build will not install	Those wheels come from a custom index.	Declare the source explicitly, or install torch outside poetry. Chapter 22.

LAB 10 – SUMMARISE-CLI, STEP 5

1. Restructure the project as a package: `src/summarise_cli/main.py` with a `main()` function.
2. Add a `[project.scripts]` entry so `summarise` is a real command, then install the project editable and run it by name.
3. Add `pytest` to a dev group and write one test that asserts your reader returns a non-empty string.
4. Run `poetry show --tree` (or `uv tree`) and find the deepest dependency you have never heard of.

Verify — typing `summarise data/input.txt` works, and `pytest` passes one test.

Where to look it up

python-poetry.org/docs – commands and configuration

packaging.python.org/specifications/pyproject-toml – the standard itself

11 Choosing between them – and migrating

Four tools, one job, and a great deal of internet argument. The honest position is that all four work, that the cost of choosing wrong is a few hours rather than a career, and that the cost of choosing nothing — installing into the system Python and hoping — is much higher. This chapter is a decision, not a debate.

The decision tree

```
Do you need non-Python system packages (CUDA toolkit, FFmpeg,
GDAL, a compiler) that pip cannot install?
├─ YES → conda / miniforge                                (ch 08)
└─ NO
    │
    ├─ Are you publishing a library to PyPI, or does
    │   packaging discipline matter more than speed?
    │   └─ YES → poetry                                    (ch 10)
    │
    ├─ Do you want one fast tool for interpreters,
    │   environments, locking and running?
    │   └─ YES → uv                                       (ch 09)
    │
    └─ Do you need the thing that is guaranteed present
        on every machine, image and CI runner you will
        ever touch – or are you following a tutorial?
        └─ YES → venv + pip                               (ch 07)
```

Figure 11.1 — Read top to bottom and stop at the first yes.

Side by side

	VENV + PIP	CONDA	UV	POETRY
Installs Python itself	No	Yes	Yes	No
Non-Python packages	No	Yes	No	No
Lockfile	Via pip-tools	<code>environment.yml</code> , weakly	<code>uv.lock</code>	<code>poetry.lock</code>
Speed	Adequate	Slow	Very fast	Slow to lock
Builds packages	No	No	Yes	Yes
Always available	Yes	No	No	No
Best at	Universality	GPU and science stacks	Everyday application work	Libraries you publish
Weak at	Reproducibility on its own	Speed, disk usage	Maturity, odd packages	Anything non-Python

GOLDEN RULE 10

One tool per project, declared in the README. Mixing two in the same repository is how a team loses a day.

Two of these coexisting on your machine is normal and fine — conda for notebook and GPU work, uv for applications, is a common and comfortable pairing. Two of them managing the *same project* is not.

Migrating without drama

FROM → TO	HOW
pip → uv	<code>uv venv</code> , then <code>uv pip install -r requirements.txt</code> . Later, <code>uv init</code> and <code>uv add</code> your direct dependencies to get a real lockfile.
pip → poetry	<code>poetry init</code> , answer the prompts, then <code>poetry add</code> each line of <code>requirements.in</code> — not <code>requirements.txt</code> , which contains transitive noise.
conda → uv/pip	Only if you have no non-Python dependencies. <code>conda list --export</code> to see the truth, then add the pip-installable ones.
poetry → uv	<code>pyproject.toml</code> is largely shared; move dependencies to <code>[project]</code> , delete <code>poetry.lock</code> , run <code>uv lock</code> .
Anything → conda	Write <code>environment.yml</code> by hand with a <code>pip:</code> section for the rest. Do not attempt automated conversion.

In every case: migrate on a branch, rebuild the environment from scratch, run your tests, and delete the old lockfile in the same commit that adds the new one. Half-migrated repositories cause more confusion than either tool alone.

When it goes wrong

SYMPTOM	CAUSE	FIX
Both <code>requirements.txt</code> and <code>poetry.lock</code> in one repo, disagreeing	Half-finished migration.	Pick one, delete the other in the same commit, note the choice in the README.
A conda environment with pip packages that vanish on rebuild	They were never recorded in <code>environment.yml</code> .	Add the <code>pip:</code> section. Chapter 08.
Teammate cannot reproduce your environment at all	You have no lockfile, only a list of names.	Chapter 13.
Tutorial commands fail because it assumes a different tool	Every tutorial assumes something.	Translate with the table above rather than switching tools mid-project.

LAB 11 – COMMIT TO ONE

1. Walk the decision tree for `summarise-cli` and write the answer, with one sentence of reasoning, at the top of its README.
2. Remove every trace of the other tools from the project: stray `environment.yml`, stray `requirements.txt`, second lockfile.
3. Delete the environment, rebuild it with your chosen tool alone, and run your test.

Verify — a stranger could read your README, run two commands, and be running your tool.

PART II — PYTHON

12 Dependency resolution: how it fails and how to read the failure

This is the most valuable chapter in Part II, because dependency errors are the ones that look like a wall of text and feel like a dead end. They are neither. Every one of them is one of six shapes, and once you can name the shape, the fix is usually two commands.

What a resolver is doing

You asked for some packages. Each of those asks for others, with version constraints. The resolver is looking for one set of versions that satisfies every constraint at once. When it cannot, it tells you — at length, because it has to explain a search. When it can, it may still have chosen versions you did not expect, which is the quieter failure mode: nothing is broken today, and your colleague gets different versions tomorrow. That second problem is what lockfiles exist for (chapter 13).

The six shapes

SHAPE	WHAT THE MESSAGE LOOKS LIKE	WHAT TO DO
No such version	<code>No matching distribution found for torch==9.9</code>	Check the real version list: <code>pip index versions torch</code> . Often a typo or a version that never existed for your platform.
Genuine conflict	<code>ResolutionImpossible</code> , then "the conflict is caused by..."	Read the "caused by" lines only. Two packages want incompatible versions of a third; relax the one you control, or drop one.
Python too new or too old	<code>requires Python >=3.9, <3.12</code> or "ignored versions"	Use a Python inside the supported range. This is why chapter 05 pins 3.11.
No wheel for your platform	Compiler errors, <code>error: command 'gcc' failed</code> , "Building wheel for..."	Do not install a compiler. Find a Python or package version with a prebuilt wheel for your architecture.
Wrong index	Package exists but pip cannot see it, or the CPU build installs instead of CUDA	Specify <code>--index-url</code> or <code>--extra-index-url</code> . PyTorch's CUDA builds are the usual case (chapter 22).
Network or TLS	<code>CERTIFICATE_VERIFY_FAILED</code> , timeouts, <code>ProxyError</code>	Not a dependency problem at all. Appendix C.

Reading a real conflict

The conflict is caused by:

```
myproject depends on numpy>=2.0
numba 0.59.1 depends on numpy<1.27
scipy 1.11.4 depends on numpy<1.28,>=1.21.6
```

To fix this you could try to:

1. loosen the range of package versions you've specified
2. remove package versions to allow pip to attempt to solve the dependency conflict

Three facts, and one of them is yours. You asked for numpy 2; numba and scipy both predate it. The fix is to relax *your* constraint to `numpy<1.27`, or to upgrade numba and scipy to releases that support numpy 2, or to drop numba if you are not using it. The generic advice at the bottom of the message is not the useful part — the three dependency lines are.

GOLDEN RULE 11

In a dependency error, find the lines naming two packages and a version. Everything else in the output is scaffolding.

The diagnostic sequence

```
$ python -V && which -a python pip      # 1. am I where I think I am?
$ python -m pip check                    # 2. is the env already broken?
$ python -m pip index versions PKG       # 3. what versions exist for me?
$ python -m pip install "PKG==X" -v      # 4. verbose: which index, which wheel
$ python -m pip show -f PKG              # 5. what is installed, and where
$ uv pip tree      # or poetry show --tree / pipdeptree
                                     # 6. who asked for this?
```

Listing 12.1 — Run these in order. Step 1 solves more cases than steps 2–6 combined.

The nuclear option, and when it is right

Delete the environment and rebuild it from the lockfile. This is not giving up; it is the correct first move whenever an environment has been hand-patched into an unknown state, and with uv it takes seconds. Reach for it early. What is *not* acceptable is deleting the environment and reinstalling by memory, because that produces a new unknown state — which is, once again, why chapter 13 exists.

When it goes wrong

SYMPTOM	CAUSE	FIX
Install "succeeds" but the wrong version lands	A constraint elsewhere forced a downgrade.	<code>pip install X</code> then <code>pip check</code> ; inspect the tree to find who pinned it.
Resolver runs for many minutes then fails	Unbounded ranges, or many candidate versions.	Pin Python, upper-bound your largest dependencies, retry.
<code>pip install</code> keeps downgrading numpy	Classic scientific-stack conflict.	Install the whole scientific stack in one command so the resolver sees all constraints at once.
Works on your machine, fails on your colleague's	Different platform, different resolved versions, no lockfile.	Chapter 13, then compare <code>pip freeze</code> output line by line.
Torch installs but reports no GPU	You got the CPU wheel.	Install from the CUDA index. Chapter 22.
Error mentions <code>metadata-generation-failed</code>	An old package with legacy packaging, or a missing build dependency.	Try an older or newer release; check whether a wheel exists at all.

LAB 12 — BREAK IT DELIBERATELY

1. In a throwaway environment, force a conflict: install an old `scipy`, then demand a `numpy` it cannot accept.
2. Read the error and write down, in one line, which two packages disagree and about what.
3. Fix it three ways: relax your constraint, upgrade the other package, remove one dependency. Note which was cleanest.
4. Delete the environment. Confirm your real project is untouched.

Verify — you can look at a `ResolutionImpossible` block and point at the three lines that matter.

PART II – PYTHON

13 Reproducible environments: pinning, lockfiles, hashes

Reproducibility is the property that the same inputs produce the same environment on another machine, or on yours next month. It is not an advanced concern for large teams. It is the difference between "my code stopped working and I do not know what changed" being a five-minute problem or a five-day one.

Four levels, increasing rigour

LEVEL	WHAT YOU HAVE	WHAT IT PROTECTS AGAINST
0	Nothing. You install as you go.	Nothing. Assume the environment is unrecoverable.
1	Names only: <code>openai</code> , <code>numpy</code>	Forgetting what you use. Not versions.
2	Ranges: <code>openai>=1.40,<2</code>	Breaking major upgrades. Not surprise minor changes.
3	A lockfile: every version of every transitive dependency	Almost everything. This is the working standard.
4	Lockfile with hashes, plus a pinned base image	Tampering and republished releases. Production and audited work.

Aim for level 3 on every project from the first day, and level 4 when something matters enough to run unattended.

The two-file pattern, per tool

TOOL	WHAT YOU WRITE	WHAT IS GENERATED	REBUILD COMMAND
pip-tools	<code>requirements.in</code>	<code>requirements.txt</code>	<code>pip install -r requirements.txt</code>
uv	<code>pyproject.toml</code>	<code>uv.lock</code>	<code>uv sync</code>
poetry	<code>pyproject.toml</code>	<code>poetry.lock</code>	<code>poetry install</code>
conda	<code>environment.yml</code>	<code>conda-lock</code> output	<code>conda env create -f ...</code>

Commit both files, always. The written one records intent; the generated one records fact. Reviewers read the first and machines read the second.

GOLDEN RULE 12

If you cannot delete your environment and rebuild it in one command, you do not have an environment — you have a pet.

Hashes, and why they are the last mile

```
$ pip-compile --generate-hashes requirements.in -o requirements.txt
$ python -m pip install --require-hashes -r requirements.txt
```

With hashes, pip refuses any file whose contents do not match the recorded digest. This closes the door on a package being replaced at the index after you locked it. It costs one flag and makes installs slightly stricter; use it anywhere the code runs without a human watching.

The rest of reproducibility

Packages are the largest part of the problem but not all of it. Four more things belong in the repository if you want a rebuild to mean anything: the Python version (`.python-version` or `requires-python`), the non-Python system dependencies (a Dockerfile, or a line in the README), the random seeds your code uses, and the data. Data is out of scope for this manual, but note that "reproducible" without a fixed dataset is a half-truth, and a checksum in the README costs nothing.

What ages and how to notice

A locked environment is frozen, not immortal: security fixes stop arriving and the wheels you pinned eventually stop building on new hardware. Update deliberately, on a schedule, on a branch, with tests — monthly is plenty for coursework. `pip list --outdated`, `uv lock --upgrade` and `poetry update` each show you the gap first.

When it goes wrong

SYMPTOM	CAUSE	FIX
Lockfile will not install on another OS	Platform-specific pins, or conda builds exported with build strings.	Use a cross-platform lock (uv, poetry, conda-lock), not <code>pip freeze</code> from one machine.
<code>pip freeze</code> output includes your own project as a path	Editable install recorded as a local path.	Normal. Do not copy that line to another machine; use the two-file pattern.
Hash mismatch on install	Package republished, or a proxy rewrote the file.	Re-lock deliberately and read what changed. Do not delete the hashes.
Lockfile conflicts on every merge	Several people re-locking independently.	One person re-locks per change; everyone else syncs. Chapter 16.
CI passes, production breaks	Different resolution, or dev dependencies present in one and not the other.	Install from the lockfile in both, with the same groups.
Rebuild works but results differ	Unseeded randomness, or different data.	Set seeds explicitly; record a dataset checksum.

LAB 13 — SUMMARISE-CLI, STEP 6

1. Make sure the project is at level 3: a written dependency file, a committed lockfile, and a pinned Python version.
2. Delete the environment. Rebuild it with one command. Time it.
3. Regenerate the lockfile with hashes and install with `--require-hashes` (or the equivalent for your tool).
4. Write the two rebuild commands into the README under a heading called *Setup*. Assume the reader is you, in a year, on a new laptop.

Verify — `rm -rf .venv && <one command> && pytest` passes from a clean state.

PART II — WHAT AGES FASTEST

Tool commands, especially uv's, and the specific version numbers in chapter 12's examples. What does not age: the PATH model in chapter 06, the six failure shapes in chapter 12, and the two-file pattern in chapter 13. If a command here has been renamed, the reasoning around it still holds — check the official docs listed at each chapter's end and update the command, not the principle.

14

PART III — TOOLS

VS Code

An editor is not where the difficulty of this work lives, so the goal of this chapter is to get VS Code out of your way in twenty minutes and never think about it again. Two things matter: that it is using the same interpreter as your terminal, and that its settings live in your repository so a teammate opening the project gets the same behaviour.

Install, per platform

PLATFORM	WITH ADMIN	WITHOUT ADMIN
macOS	Drag to Applications; then <i>Shell Command: Install 'code'</i>	Same — no admin required
Linux	<code>apt</code> repo or <code>.deb</code>	The <code>.tar.gz</code> extracted into your home directory
Windows	System installer	User installer (default) or the portable ZIP
WSL2	Install on Windows, then open projects with <code>code .</code> from inside a WSL2 shell	

The five extensions worth having

Python (the official Microsoft one, which brings Pylance), Jupyter, Docker, Remote-SSH, and Ruff. That is the whole list. Resist the urge to install twenty more; each one is a background process and a source of confusing behaviour, and an editor that starts in eight seconds instead of two is a tax you pay all day.

Selecting the interpreter — the one thing to get right

Open the command palette (Cmd/Ctrl-Shift-P), run *Python: Select Interpreter*, and choose the one inside `./.venv`. The chosen path appears in the status bar; check it whenever imports behave strangely. If your project's environment is not offered, open the project folder itself rather than a parent directory — VS Code looks for `.venv` at the workspace root.

```
.vscode/settings.json
{
  "python.defaultInterpreterPath": "${workspaceFolder}/.venv/bin/python",
  "python.terminal.activateEnvironment": true,
  "editor.formatOnSave": true,
  "editor.rulers": [88],
  "files.exclude": { "**/__pycache__": true, "**/.pytest_cache": true },
  "[python]": { "editor.defaultFormatter": "charliermarsh.ruff" }
}
```

Listing 14.1 — Commit this file. On Windows the interpreter path ends `\.venv\Scripts\python.exe`.

GOLDEN RULE 13

If the editor and the terminal disagree about your Python, believe the terminal and fix the editor.

Shortcuts that pay for themselves in a week

SHORTCUT	DOES
Cmd/Ctrl-Shift-P	Command palette — everything is here, searchable
Cmd/Ctrl-P	Open any file by typing part of its name
Cmd/Ctrl-Shift-F	Search the whole project
Ctrl-`	Toggle the integrated terminal
F12 / Alt-F12	Go to definition / peek at it without leaving
F2	Rename a symbol everywhere it is used
F5 / F9	Start the debugger / set a breakpoint
Cmd/Ctrl-Shift-M	Problems panel — every error the language server sees

The debugger, briefly, because it replaces print

Set a breakpoint by clicking left of a line number, press F5, and choose *Python File*. Execution stops at the line and you can inspect every variable in scope, step through, and evaluate expressions in the debug console. Ten minutes learning this saves more time than any other ten minutes in this manual, and it is the habit that most reliably separates people who debug from people who guess.

When it goes wrong

SYMPTOM	CAUSE	FIX
Imports underlined in red but the code runs	Language server using a different interpreter.	Select the interpreter; reload the window.
Code runs in the terminal but not with the Run button	Run uses the selected interpreter and a different working directory.	Select the right interpreter; set <code>"cwd"</code> in a launch configuration.
Terminal does not activate the environment	Setting off, or a shell that VS Code cannot hook.	<code>python.terminal.activateEnvironment: true</code> , or activate manually.
Everything is slow, fans spinning	Indexing a huge folder — often <code>.venv</code> , <code>node_modules</code> or a dataset.	Add them to <code>files.watcherExclude</code> ; open the project folder, not your home directory.
On WSL2, extensions seem missing	Extensions install per-context; WSL2 is its own context.	Install them again inside the WSL2 window. This is expected.
Formatting fights you on save	Two formatters enabled.	Set one <code>defaultFormatter</code> per language, as in Listing 14.1.

LAB 14 — SUMMARISE-CLI, STEP 7

1. Open the project folder, select the `.venv` interpreter, and confirm the status bar shows it.
2. Create `.vscode/settings.json` from Listing 14.1 and commit it.
3. Set a breakpoint in your file-reading function, run under the debugger, and inspect the variable holding your text.
4. Fix one thing the Problems panel is complaining about.

Verify — the debugger stops at your breakpoint and shows your text in the Variables panel.

15 Git: the model, then the commands

Git is worth understanding rather than memorising, because a memorised sequence fails the moment reality deviates from the tutorial. The model is small: git stores snapshots of your whole project, each with a parent, forming a graph. Everything else is a way of adding to that graph or moving a pointer around it.

Four places a file can be

```
working tree  —git add→  staging area  —git commit→  local repo  —git
push→  remote
  (your edits)                (what will go in          (permanent
                             the next commit)          history)
```

Almost every confusing git moment is not knowing which of those four holds the version you care about. `git status` answers it, every time, and is the command you should run before and after everything else until it becomes reflex.

One-time setup

```
$ git config --global user.name "Your Name"
$ git config --global user.email "you@example.com"
$ git config --global init.defaultBranch main
$ git config --global pull.rebase false      # merge on pull: predictable for
beginners
$ git config --global core.autocrlf input    # Windows only: stop line-ending churn
```

The daily commands

COMMAND	WHAT IT DOES
<code>git status</code>	Where everything stands. Run it constantly.
<code>git add -p</code>	Stage changes one hunk at a time — read your own work before committing
<code>git commit -m "message"</code>	Snapshot what is staged
<code>git log --oneline --graph --decorate -20</code>	The last twenty commits, with the branch shape visible
<code>git diff</code> / <code>git diff --staged</code>	Unstaged changes / staged changes
<code>git switch -c feature/x</code>	New branch, and move onto it
<code>git switch main</code>	Move between branches
<code>git pull</code> / <code>git push -u origin feature/x</code>	Sync down / publish a branch
<code>git restore file</code>	Throw away uncommitted changes to a file
<code>git restore --staged file</code>	Unstage without losing the edit
<code>git stash</code> / <code>git stash pop</code>	Park uncommitted work, get it back
<code>git revert <sha></code>	Undo a commit with a new commit — safe on shared branches
<code>git reset --soft HEAD~1</code>	Undo the last commit, keep the changes staged
<code>git reflog</code>	Every position HEAD has held. This is how you recover "lost" work.
<code>git blame file</code>	Who last touched each line, and in which commit
<code>git bisect start</code>	Binary-search history for the commit that broke something

GOLDEN RULE 14

Commit early and often on your own branch. A commit is a save point, not a publication — and `git reflog` means almost nothing committed is ever truly lost.

.gitignore, written once

```
.venv/  
__pycache__/  
*.pyc  
.env  
.env.*  
!.env.example  
.ipynb_checkpoints/  
data/raw/  
models/  
*.ckpt  
*.safetensors  
.DS_Store  
.vscode/*  
!.vscode/settings.json
```

Listing 15.1 — A starting `.gitignore` for AI work. Environments, secrets, data and weights never belong in git; the shared editor settings do.

Commit messages that will help you later

A subject line under about seventy characters, in the imperative — "add retry logic to summariser", not "added" or "adding". If the change needs explanation, leave a blank line and write why, not what: the diff already says what. You are writing for yourself in six months, trying to work out why this line exists.

When it goes wrong

SYMPTOM	CAUSE	FIX
Committed a secret	It was not in <code>.gitignore</code> .	Rotate the key first — assume it is public. Then rewrite history. Chapter 27.
Committed a 2 GB model file, push rejected	GitHub's file size limit.	Remove it from history (<code>git filter-repo</code>), add it to <code>.gitignore</code> , use Git LFS or object storage.
Please tell me who you are	No <code>user.email</code> configured.	The setup block above.
Merge conflict markers in a file	Two branches changed the same lines.	Edit the file, delete the markers, keep the right code, <code>git add</code> , <code>git commit</code> . Chapter 16.
Detached HEAD	You checked out a commit rather than a branch.	<code>git switch -c fix-branch</code> to keep the work, or <code>git switch main</code> to abandon it.
"I lost my work"	Almost never true.	<code>git reflog</code> , find the sha, <code>git switch -c recovered <sha></code> .
Notebook diffs are unreadable	Notebooks are JSON with embedded output.	Strip output before committing. Chapter 20.
Whole files show as changed on Windows	Line endings.	<code>core.autocrlf input</code> , plus a <code>.gitattributes</code> .

LAB 15 — SUMMARISE-CLI, STEP 8

1. `git init` the project, add the `.gitignore` from Listing 15.1, and make your first commit.
2. Confirm `git status` shows no `.venv` and no `.env`. If it does, fix the ignore file before continuing.
3. Make a branch, change one line, commit, switch back, and look at `git log --graph` to see the shape.
4. Deliberately delete a committed file, then restore it from history. Then `git reflog` and read what it recorded.

Verify — `git log --oneline` shows at least three commits, and no ignored file has ever been committed.

PART III — TOOLS

16 GitHub workflows: branches, pull requests, review

Git is the tool; a workflow is the agreement about how a group uses it. For a cohort or a small team, one agreement covers almost everything: `main` always works, all work happens on short-lived branches, and every branch arrives on `main` through a pull request that someone else has read.

The loop

```
$ git switch main && git pull           # start from current main
$ git switch -c feature/retry-logic     # small, named for its purpose
... work, commit two or three times ...
$ git push -u origin feature/retry-logic
... open a pull request, describe it, request a review ...
... address comments with more commits ...
... squash-merge, delete the branch ...
$ git switch main && git pull           # and again
```

Listing 16.1 — A branch that lives less than two days rarely produces a hard conflict.

Authenticating, once

Use SSH keys (chapter 17) or the GitHub CLI. Password authentication for git operations no longer exists, and a personal access token typed into a prompt every time is a poor substitute for a key.

```
$ gh auth login           # interactive, handles both HTTPS and SSH
$ gh repo clone owner/repo
$ gh pr create --fill     # from your current branch
$ gh pr checkout 42       # review someone else's work locally
$ gh pr status
```

What a good pull request contains

- A title that says what changes, not which files change.
- Two or three sentences on why, and what you decided against.
- How to verify it — the exact command a reviewer should run.
- Under about four hundred changed lines. Beyond that, review quality collapses and you get approval without reading.
- No unrelated formatting churn. Reformat in its own commit, or its own PR.

GOLDEN RULE 15

`main` is always in a state you would be happy to demonstrate. Everything unfinished lives on a branch.

Resolving a conflict, calmly

```
$ git switch feature/x
$ git pull origin main          # bring main's changes into your branch
... git reports conflicts ...
$ git status                   # which files
... open each, remove <<<<, ==, >>>> markers, keep the right code ...
$ git add <file> && git commit
$ git push
```

Conflicts are not a failure of skill; they are two people editing the same lines. Resolve them on *your* branch, never by force-pushing over someone else's, and if a conflict is in a lockfile, regenerate it rather than hand-editing it.

Reviewing someone else's work

Read the description first, then the tests, then the code. Ask questions rather than issuing instructions — "what happens if this file is empty?" lands better and teaches more than "add a check here". Approve when it is better than what is on `main`, not when it is perfect; perfect blocks everything forever. And say what you liked: review is the main channel through which a cohort's standards actually spread.

When it goes wrong

SYMPTOM	CAUSE	FIX
Permission denied (publickey)	No SSH key registered with GitHub.	Chapter 17.
Updates were rejected because the remote contains work you do not have	Someone pushed since you last pulled.	<code>git pull</code> , resolve, push. Never <code>--force</code> on a shared branch.
Your PR shows hundreds of unrelated changes	Branched from a stale main, or a reformat crept in.	Rebase onto current main, or open a fresh branch and cherry-pick your commits.
Reviewer cannot reproduce your result	Environment or data differences.	Lockfile plus setup commands in the PR body. Chapter 13.
Merge button greyed out	Failing checks or unresolved review.	Read the checks — that is chapter 29 doing its job.
Force-pushed over a colleague's commits	<code>--force</code> on a shared branch.	Their <code>git reflog</code> still has it. Recover, then use <code>--force-with-lease</code> from now on.

LAB 16 — SUMMARISE-CLI, STEP 9

1. Push the project to a new GitHub repository. Write a README with a one-line description and the setup commands from Lab 13.
2. Open a pull request that adds one feature — a `--max-words` option — with a description a stranger could act on.
3. Have someone review it, or review a classmate's. Leave one question and one specific compliment.
4. Create a conflict on purpose with a classmate, then resolve it. Do this once now, while it is cheap.

Verify — a merged pull request exists, `main` runs, and the branch is deleted.

Where to look it up

docs.github.com/pull-requests — review mechanics and protections

git-scm.com/book — the reference for the model, free and good

17 SSH keys, remote development, dev containers

Sooner or later the machine doing the work is not the machine you are typing on: a rented GPU, a lab server, a colleague's box. SSH is how you get there, and VS Code's remote support means you can edit files on that machine as if they were local, with your own editor and extensions.

Generate a key once, use it everywhere

```
$ ssh-keygen -t ed25519 -C "you@example.com"      # accept the default path
$ cat ~/.ssh/id_ed25519.pub                      # the PUBLIC half – safe to share
$ ssh-add ~/.ssh/id_ed25519                     # load into the agent
$ ssh -T git@github.com                          # test: expect a greeting by name
```

Give the key a passphrase. The agent remembers it for your session, so you type it once a day rather than once a push. The file without `.pub` is the private half: it never leaves your machine, never goes in a repository, never gets pasted into a chat.

Add the public key to GitHub under Settings → SSH and GPG keys. One key per machine, named for the machine, so you can revoke a lost laptop's access without disturbing anything else.

~/.ssh/config – the file that makes this pleasant

```
Host gpu
  HostName 203.0.113.42
  User ubuntu
  IdentityFile ~/.ssh/id_ed25519
  ServerAliveInterval 60      # survive idle NAT timeouts

Host github.com               # when port 22 is blocked by a firewall
  Hostname ssh.github.com
  Port 443
  User git
```

Listing 17.1 — With this file, `ssh gpu` is the whole command. The second block is the standard fix for corporate networks (Appendix C).

Moving files, and forwarding ports

```
$ scp local.txt gpu:~/data/           # one file up
$ rsync -avP ./data/ gpu:~/data/      # resumable, skips unchanged – use
for datasets
$ ssh -L 8888:localhost:8888 gpu      # reach a remote Jupyter in your
local browser
```

VS Code Remote-SSH

With the Remote-SSH extension, the command palette's *Connect to Host* reads your `~/.ssh/config` and opens a window whose files, terminal and extensions all live on the remote machine. Your keys stay local; nothing is copied. This is the difference between using a rented GPU comfortably and dreading it.

One caution: everything you install in that window installs on the remote machine, and when the machine is destroyed it all goes with it. Keep the project in git and the environment in a lockfile, and rebuilding on a fresh box is two commands (chapter 13).

Dev containers

A dev container is a Docker image described in your repository that VS Code opens as your development environment. Everyone on the project gets identical tools regardless of their laptop, which is worth a great deal on a team and rather a lot of ceremony for one person. It needs Docker, so it is unavailable without admin rights.

```
.devcontainer/devcontainer.json
{
  "name": "aie",
  "image": "mcr.microsoft.com/devcontainers/python:3.11",
  "postCreateCommand": "uv sync",
  "customizations": { "vscode": { "extensions": ["ms-python.python",
"charliermarsh.ruff"] } }
}
```

GOLDEN RULE 16

The private key never moves. Copy the public half anywhere; if a private key has ever been on a shared machine or in a repository, replace it.

When it goes wrong

SYMPTOM	CAUSE	FIX
Permission denied (publickey)	Key not registered, or not offered by the agent.	<code>ssh-add -l</code> to list loaded keys; <code>ssh -vT git@github.com</code> to see which are tried.
Connection timed out to port 22	Firewall blocking SSH.	The <code>ssh.github.com:443</code> block in Listing 17.1. Appendix C.
WARNING: UNPROTECTED PRIVATE KEY FILE	Permissions too open.	<code>chmod 600 ~/.ssh/id_ed25519</code>
REMOTE HOST IDENTIFICATION HAS CHANGED	The server was rebuilt — normal for rented GPUs.	Remove the old line from <code>~/.ssh/known_hosts</code> . Verify it really was rebuilt first.
Connection drops when idle	NAT or firewall timeout.	<code>ServerAliveInterval 60</code> ; run long jobs under tmux (chapter 18).
Remote VS Code very slow	Extensions indexing a large remote dataset.	Exclude data directories; keep datasets outside the workspace folder.
Passphrase asked on every push	No agent running.	<code>ssh-add</code> ; on macOS add <code>UseKeychain yes</code> to the config.

LAB 17 – KEYS AND A REMOTE

1. Generate an ed25519 key with a passphrase, add the public half to GitHub, and switch your project's remote to SSH.
2. Write a `~/.ssh/config` entry for GitHub, including the port-443 fallback, and test with `ssh -T`.
3. If you have access to any remote machine — a rented GPU, a classmate's box, a free tier VM — connect with Remote-SSH and run your project's tests there.

Verify — `git push` works with no password prompt, and `ssh -T git@github.com` greets you by username.

18

PART III – TOOLS

Shell productivity: aliases, dotfiles, tmux

This chapter is about the difference between typing a hundred characters and typing four. None of it is necessary; all of it compounds. Add one thing a week rather than everything today — a shell configuration you do not understand is a liability, not a convenience.

Aliases and functions worth having

```
# in ~/.zshrc or ~/.bashrc
alias ll='ls -lah'
alias gs='git status -sb'
alias gd='git diff'
alias gp='git pull'
alias venv='source .venv/bin/activate'
alias pipi='python -m pip install'

# make a directory and enter it
mkcd() { mkdir -p "$1" && cd "$1"; }

# activate .venv walking up from anywhere in a project
act() { d=$PWD; while [ "$d" != / ]; do [ -f "$d/.venv/bin/activate" ] && source "$d/.venv/bin/activate" && return; d=$(dirname "$d"); done; echo "no .venv found"; }
```

Listing 18.1 — PowerShell equivalents go in the file that `$PROFILE` names; the syntax differs but the idea is identical.

Dotfiles: your environment as a repository

Your shell configuration, git configuration, editor settings and SSH config are all just text files, and putting them in a git repository turns "set up a new machine" from an afternoon of remembering into ten minutes of copying. Keep them in `~/dotfiles`, symlink them into place, and never put a secret in there — the whole point is that this repository can be public, and the moment it holds a key it cannot be.

```
$ git init ~/dotfiles && cd ~/dotfiles
$ mv ~/.zshrc .zshrc && ln -s ~/dotfiles/.zshrc ~/.zshrc
$ mv ~/.gitconfig .gitconfig && ln -s ~/dotfiles/.gitconfig ~/.gitconfig
$ git add -A && git commit -m "start dotfiles"
```

tmux: the reason a dropped connection stops mattering

When you close an SSH session, everything running in it dies. For a two-hour training run over hotel wifi that is unacceptable. tmux keeps a session alive on the remote machine whether or not you are attached, so you can disconnect, close your laptop, and reattach later to find your job still running.

COMMAND / KEYS	DOES
<code>tmux new -s train</code>	Start a named session
Ctrl-b then d	Detach — the session keeps running
<code>tmux ls</code> / <code>tmux attach -t train</code>	List sessions / come back
Ctrl-b then c / n / p	New window, next, previous
Ctrl-b then % / "	Split vertically / horizontally
Ctrl-b then arrow keys	Move between panes
Ctrl-b then [	Scroll back through output; <code>q</code> to exit
<code>tmux kill-session -t train</code>	End it

Ctrl-b is the prefix: every tmux command starts with it. That is the only thing standing between most people and using tmux, and it takes an afternoon to stop being awkward.

GOLDEN RULE 17

Any job longer than your patience runs inside tmux. Anything else is a bet on your network.

Four more small things

- `fzf` — fuzzy search over history and files. Ctrl-R becomes genuinely powerful.
- `ripgrep` (`rg`) — search a large codebase in the time `grep` takes to start.
- `bat`, `eza` — `cat` and `ls` with syntax highlighting and sensible defaults.
- `direnv` — loads a project's environment variables when you `cd` into it, and unloads them when you leave. Pairs well with chapter 27.

When it goes wrong

SYMPTOM	CAUSE	FIX
New terminal windows are slow to open	A heavy prompt, plugin manager, or conda init in your rc file.	Comment out half, reopen, bisect until you find it.
An alias shadows a real command	You named it after something that exists.	<code>type name</code> shows what it resolves to; <code>\name</code> or <code>command name</code> bypasses the alias.
Edits to <code>.zshrc</code> do nothing	Not reloaded, or your shell reads <code>.bash_profile</code> instead.	<code>source ~/.zshrc</code> ; confirm your shell with <code>echo \$0</code> .
tmux session gone after a reboot	Sessions do not survive reboots.	Expected. For unattended long jobs use a scheduler or a service.
Colours or arrow keys broken inside tmux	<code>TERM</code> mismatch.	Add <code>set -g default-terminal "screen-256color"</code> to <code>~/.tmux.conf</code> .
Dotfiles repo contains a token	You committed <code>.env</code> or a config with a key.	Rotate the key, rewrite history, make the repo private. Chapter 27.

LAB 18 — FOUR KEYSTROKES INSTEAD OF FORTY

1. Add three aliases and the `act` function to your shell profile. Reload and use them for a day before adding more.
2. Create a `~/dotfiles` repository holding your shell and git configuration, symlinked into place. Confirm no secrets are in it.
3. Start a tmux session, run a command that takes a minute, detach, close the terminal entirely, reopen and reattach.

Verify — you can reattach to a running job after closing your terminal, and a fresh clone of your dotfiles would configure a new machine.

19 Makefiles, task runners, pre-commit hooks

Every project accumulates a handful of commands that everyone needs and nobody remembers: how to install, how to test, how to format, how to run. Writing them down in an executable file is the cheapest documentation there is — it cannot go stale without failing loudly.

A Makefile as a table of contents

```
.PHONY: setup test fmt lint run clean

setup:          ## create the environment from the lockfile
    uv sync

test:           ## run the test suite
    uv run pytest -q

fmt:            ## format and fix imports
    uv run ruff format . && uv run ruff check --fix .

lint:           ## check without changing anything
    uv run ruff check .

run:            ## summarise the sample document
    uv run summarise data/input.txt

clean:          ## remove the environment and caches
    rm -rf .venv .pytest_cache */__pycache__
```

Listing 19.1 — Recipe lines must begin with a real tab character, not spaces. This is Make's one cruelty and it catches everyone once.

Now `make setup`, `make test` and `make run` are the interface to the project, on any machine, forever. A new person needs to read six lines rather than a page of prose.

Windows has no `make` by default. Either work in WSL2, or use a cross-platform runner: `just` (a `justfile`, near-identical syntax, no tab rule), `nox`, or plain scripts in a `scripts/` folder. Whichever you choose, commit it and name it in the README.

Pre-commit hooks: the errors you never have to discuss

A pre-commit hook runs checks before a commit is created and refuses the commit if they fail. It converts a category of review comment — formatting, trailing whitespace, a stray debugger, a committed secret — into something a machine handles silently and nobody has to be tactful about.

```
.pre-commit-config.yaml
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.6.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-added-large-files      # stops the 2 GB model commit
        args: ["--maxkb=5000"]
      - id: check-merge-conflict
      - id: detect-private-key
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.6.9
    hooks:
      - id: ruff
        args: [--fix]
      - id: ruff-format
  - repo: https://github.com/kynan/nbstripout
    rev: 0.7.1
    hooks:
      - id: nbstripout                  # notebook output, chapter 20
```

```
$ uv tool install pre-commit      # or pipx / pip --user
$ pre-commit install              # activate for this clone – everyone must run
this once
$ pre-commit run --all-files      # first run: fixes the whole repo
$ pre-commit autoupdate           # bump the rev pins occasionally
```

GOLDEN RULE 18

Automate the boring checks and let review spend its attention on design. Never argue about formatting; make a tool decide.

Formatting and linting, settled

Use `ruff` for both. It formats compatibly with the long-standing community style and lints fast enough to run on every save. Configure it once in `pyproject.toml` and never discuss it again.

```
[tool.ruff]
line-length = 88
target-version = "py311"

[tool.ruff.lint]
select = ["E", "F", "I", "UP", "B"]  # errors, pyflakes, imports, upgrades,
bugbear
```


When it goes wrong

SYMPTOM	CAUSE	FIX
<code>Makefile:4: *** missing separator</code>	Spaces instead of a tab.	Replace the leading whitespace with one tab. Configure your editor to keep tabs in Makefiles.
<code>make: command not found</code> on Windows	Not installed.	WSL2, or <code>just</code> , or scripts.
Hooks do not run	<code>pre-commit install</code> not run in this clone.	Run it. Hooks live in <code>.git/</code> , which is never committed.
A hook blocks an urgent commit	It is doing its job.	Fix the issue. <code>--no-verify</code> exists; using it routinely defeats the point.
Hooks reformat files mid-commit and the commit fails	Normal: they fixed the files, so the commit must be remade.	<code>git add -u</code> and commit again.
CI and local formatting disagree	Different tool versions.	Pin <code>rev:</code> in the config and the same version in CI. Chapter 29.

LAB 19 – SUMMARISE-CLI, STEP 10

1. Add the Makefile from Listing 19.1 (or a `justfile`) and make every target work.
2. Add the pre-commit configuration, install the hooks, and run them over the whole repository. Commit the fixes.
3. Try to commit a file with trailing whitespace and a fake private key. Watch both get stopped.
4. Replace the setup instructions in your README with `make setup && make test`.

Verify — `make test` passes from a clean clone, and a badly formatted commit is refused.

20 Jupyter: notebook hygiene and notebook-to-script

Notebooks are excellent for exploring and poor for building. Both halves of that sentence matter: dismissing notebooks costs you the fastest way to look at data, and shipping them costs you reproducibility, review and testing. The professional habit is to explore in a notebook and move anything that works into a module.

Run it from your project environment

```
$ uv add --dev jupyterlab ipykernel
$ uv run jupyter lab
$ # or register this environment as a named kernel for a central Jupyter
$ uv run python -m ipykernel install --user --name summarise-cli --display-name
"summarise-cli (3.11)"
```

The kernel you pick inside a notebook is an interpreter choice exactly like chapter 06's, and the failure mode is the same: a notebook running a different Python than your terminal, with different packages. Check it explicitly.

```
>>> import sys; print(sys.executable)    # first cell of every notebook, honestly
```

GOLDEN RULE 19

A notebook that only works when run out of order is broken. Restart and run all before you believe a result or share it.

Hygiene, in six habits

- **Restart and Run All before trusting output.** Hidden state — a variable from a cell you have since edited — is the defining notebook bug.
- **Strip output before committing.** `nbstripout` as a pre-commit hook (chapter 19) makes diffs readable and keeps large images and, occasionally, printed API keys out of your history.
- **Keep imports and configuration in the first cell.** Everything else can then be run in order.
- **No secrets in cells.** Load them from the environment (chapter 27); a printed key lives in the output forever.
- **Move working code out.** When a function is right, put it in `src/` and import it. `%load_ext autoreload` and `%autoreload 2` mean you keep the fast loop without keeping the code in the notebook.

- **Name notebooks in order.** `01-explore-data.ipynb`, `02-baseline.ipynb`. A folder of `Untitled7.ipynb` is a folder of lost work.

Notebook to script

```
$ jupyter nbconvert --to script 01-explore.ipynb      # quick, messy starting point
$ uv run jupyter --set-formats ipynb,py:percent nb.ipynb  # paired files,
diffable
$ uv run papermill nb.ipynb out.ipynb -p max_words 200    # run with parameters
```

The direction of travel is one-way: exploration lives in notebooks, and anything that runs twice becomes a function in a module with a test. By the end of a project the notebooks should be thin — a few cells that import your own package and draw a chart.

When it goes wrong

SYMPTOM	CAUSE	FIX
<code>ModuleNotFoundError</code> in a notebook, but the package is installed	The kernel is a different interpreter.	Print <code>sys.executable</code> ; register and select the project kernel.
Works for you, fails for everyone else	Out-of-order execution and hidden state.	Restart and Run All. Read the execution counts.
Notebook diff is thousands of lines	Embedded output and metadata.	<code>nbstripout</code> , or jupyter text pairing.
Kernel dies repeatedly	Out of memory — usually a large dataframe or model.	Load a sample; delete large objects; restart. Watch memory while it runs.
An API key appears in committed output	You printed a variable, or a traceback showed it.	Rotate the key immediately, then strip and rewrite history. Chapter 27.
Cannot import your own <code>src/</code> package	The notebook's working directory is not the project root.	Install the project editable (<code>uv pip install -e .</code>) rather than editing <code>sys.path</code> .
Notebook takes minutes to open	Megabytes of stored output.	Strip it. Save large artifacts to disk instead.

LAB 20 – SUMMARISE-CLI, STEP 11

1. Create `notebooks/01-explore.ipynb` in the project environment. First cell prints `sys.executable` and confirms it is your `.venv`.
2. Explore your input text: length, word frequencies, a chart. Then move the two functions you liked into `src/summarise_cli/` and import them back.
3. Enable `nbstripout`, commit the notebook, and confirm the diff is small.
4. Restart and Run All. If anything breaks, that is the lesson.

Verify — the notebook runs top to bottom in a fresh kernel, and its committed diff contains no output.

PART III – WHAT AGES FASTEST

Extension names, the specific linters in fashion, and the `rev:` pins in Listing 19.1 — those are meant to be updated, and `pre-commit autoupdate` does it for you. Git's model, the pull-request loop, SSH, tmux and the notebook discipline in chapter 20 will outlast several editors.

21 PART IV — COMPUTE Google Colab and free GPU

Colab is a notebook running on someone else's machine, with a GPU attached if you ask for one, for free within limits. For a cohort where some people have no GPU and some have no admin rights, it is the great leveller — and its constraints are educational in themselves, because they are the constraints of all rented compute.

What it is, precisely

A temporary virtual machine with a preinstalled Python environment, an attached accelerator, and a disk that is erased when the session ends. Nothing you install and nothing you save persists unless you deliberately put it somewhere else. Sessions also end on their own: after a period of inactivity, or when you hit a usage limit that is not published and varies with demand.

The first four cells of any Colab notebook

```
# 1 — what did I get?
!nvidia-smi
import torch; print(torch.__version__, torch.cuda.is_available())

# 2 — mount Drive so results survive the session
from google.colab import drive; drive.mount('/content/drive')

# 3 — secrets, from Colab's own store, never pasted in a cell
from google.colab import userdata
import os; os.environ["OPENAI_API_KEY"] = userdata.get("OPENAI_API_KEY")

# 4 — your code, from git, not copy-pasted
!git clone https://github.com/<you>/summarise-cli.git
%cd summarise-cli
!pip install -q -r requirements.txt
```

Listing 21.1 — Cell 3 uses the Secrets panel (the key icon in the sidebar). A key typed into a cell is a key in your notebook's saved output.

GOLDEN RULE 20

Treat every Colab session as disposable. Code comes from git, secrets come from the Secrets panel, results go to Drive or object storage — before the session dies, not after.

Living with the limits

LIMIT	WHAT HAPPENS	HOW TO WORK WITH IT
Idle timeout	The runtime disconnects after inactivity	Checkpoint every few minutes; save to Drive as you go
Maximum session length	Runtime ends mid-job	Make jobs resumable from a checkpoint. Never rely on a single long run.
GPU availability	You may be given none, or a slower one	Check cell 1 every time; write code that runs on CPU too, slowly
Disk is temporary	<code>/content</code> is erased	Drive for small outputs; re-download weights each session (chapter 26)
Installs do not persist	Reinstall every session	One <code>pip install -r</code> cell. This is a reproducibility test you get for free.
Preinstalled versions drift	A notebook that worked last month fails	Pin the versions you depend on in <code>requirements.txt</code>

Colab habits that transfer

Everything in that table is also true of a rented GPU (chapter 23) and largely true of a CI runner (chapter 29): ephemeral machine, code from git, secrets from a store, results written somewhere permanent, work resumable from a checkpoint. Learning to work this way in Colab is not a compromise; it is practice for every remote machine you will ever use.

When it goes wrong

SYMPTOM	CAUSE	FIX
<code>torch.cuda.is_available()</code> is False	No accelerator selected, or none available.	Runtime → Change runtime type → GPU. Then rerun cell 1.
Runtime disconnects repeatedly	Idle timeout, or usage limits.	Interact periodically; checkpoint; consider a paid tier or a rented box.
CUDA out of memory	Model or batch too large for the GPU you were given.	Smaller batch, gradient accumulation, a quantised model. Chapter 22.
Everything vanished	Session ended; <code>/content</code> is gone.	Expected. Nothing to recover. Save earlier next time.
Drive mount fails or hangs	Permission prompt not completed, or a stale mount.	Rerun the mount cell; <code>drive.flush_and_unmount()</code> then remount.
Reading many small files from Drive is glacial	Drive is network storage.	Copy an archive to <code>/content</code> and unpack it there.
Notebook works in Colab, fails locally	Colab's preinstalled packages differ from yours.	Install from your <code>requirements.txt</code> in cell 4 so both machines match.

LAB 21 — SUMMARISE-CLI, STEP 12

1. Open a Colab notebook with a GPU runtime and run cell 1. Write down which GPU you were given and how much memory it has.
2. Put your API key in the Secrets panel and load it as in cell 3. Confirm nothing sensitive appears in any output.
3. Clone your repository, install from the lockfile, and run your tool in Colab. Fix whatever does not work — that is your reproducibility gap.
4. Save the output to Drive, then deliberately end the runtime and confirm the file survived.

Verify — your tool runs unchanged in Colab from a fresh session, in under five cells.

PART IV — COMPUTE

22 GPUs, drivers and CUDA: the compatibility matrix

GPU setup has a reputation for misery, and it is deserved, but the misery has exactly one cause: four layers that must agree with each other, and error messages that never say which layer is wrong. Learn the layers and the problem becomes tedious rather than mysterious.

The four layers, bottom to top

- | | |
|--------------------------|--|
| 4. Your Python package | torch 2.4.0+cu121, jax, tensorflow |
| ▲ must be built for | _____ |
| 3. CUDA runtime | bundled inside the wheel, usually |
| ▲ must be ≤ supported by | _____ |
| 2. NVIDIA driver | installed on the HOST operating system |
| ▲ must support | _____ |
| 1. The GPU itself | compute capability, VRAM |

Two facts remove most of the confusion. First, modern PyTorch wheels *bundle* their own CUDA runtime, so you generally do not install the CUDA toolkit at all — you install the right wheel. Second, the driver is a host concern: on WSL2 you install the Windows driver and nothing inside Linux; in Docker the driver is on the host and the container brings the rest.

Diagnose in three commands

```
$ nvidia-smi                                # layer 2: driver present? which version?
what is using memory?
$ python -c "import torch; print(torch.__version__, torch.version.cuda,
torch.cuda.is_available(), torch.cuda.device_count())"
$ python -c "import torch; print(torch.cuda.get_device_name(0))"
```

`nvidia-smi` works but torch says False: layer 3 or 4 — you have the CPU wheel, or a runtime newer than your driver supports. `nvidia-smi` not found: layer 2 — driver, or you are inside WSL2 or a container looking for something that lives on the host.

Installing the right build

```
$ # NVIDIA GPU: pick the CUDA build from PyTorch's own index
$ uv pip install torch --index-url https://download.pytorch.org/whl/cu121

$ # CPU only — smaller download, works everywhere
$ uv pip install torch --index-url https://download.pytorch.org/whl/cpu

$ # Apple Silicon: the default wheel, and use the mps device
$ uv pip install torch
$ python -c "import torch; print(torch.backends.mps.is_available())"
```

Listing 22.1 — Always check the current install selector on pytorch.org: the `cu###` suffix moves with each release, and this page will age faster than any other in the manual.

Write code that runs anywhere

```
import torch
device = ("cuda" if torch.cuda.is_available()
          else "mps" if torch.backends.mps.is_available()
          else "cpu")
model.to(device)
```

Three lines, and your work runs on a lab GPU, a MacBook and a CPU-only laptop without editing. Do this from the first project; hard-coding `cuda` is how a script becomes unshareable.

GOLDEN RULE 21

Do not install the CUDA toolkit to make PyTorch work. Install the wheel built for your driver. The toolkit is for compiling CUDA code, which is not what you are doing.

The compatibility matrix

SITUATION	WHAT MUST BE TRUE	WHAT YOU INSTALL
Linux + NVIDIA	Driver recent enough for the wheel's CUDA	Distro driver package, then the <code>cu###</code> torch wheel
Windows + NVIDIA, native	Windows driver installed	The <code>cu###</code> wheel in a Windows environment
WSL2 + NVIDIA	Driver on <i>Windows</i> , nothing in Linux	The <code>cu###</code> wheel inside WSL2
Docker + NVIDIA	Host driver, plus NVIDIA Container Toolkit	<code>docker run --gpus all</code> , CUDA base image. Chapter 25.
Apple Silicon	Nothing — it is built in	Default torch wheel; use <code>mps</code>
No GPU	Nothing	CPU wheel; smaller models; Colab for the rest

When it goes wrong

SYMPTOM	CAUSE	FIX
<code>torch.cuda.is_available()</code> is False, <code>nvidia-smi</code> fine	CPU wheel installed.	<code>print(torch.version.cuda)</code> — if <code>None</code> , reinstall from the CUDA index.
CUDA driver version is insufficient for CUDA runtime version	Wheel's runtime newer than the driver.	Update the driver, or install a wheel with a lower <code>cu###</code> .
CUDA out of memory	VRAM exhausted — not system RAM.	Smaller batch; <code>torch.cuda.empty_cache()</code> ; check for another process in <code>nvidia-smi</code> ; quantise.
Works, then fails after a system update	Driver replaced.	Recheck the matrix; reinstall the matching wheel.
no kernel image is available for execution	Wheel not built for your GPU's compute capability — usually a very new or very old card.	A newer wheel, a nightly build, or a different machine.
Multiple GPUs, code uses one	Default is device 0.	<code>CUDA_VISIBLE_DEVICES=0,1</code> , and a parallel strategy in code.
Apple Silicon: an operation is unimplemented on mps	Coverage gaps remain.	<code>PYTORCH_ENABLE_MPS_FALLBACK=1</code> to fall back to CPU for that op.
GPU idle while the job crawls	Data loading is the bottleneck, not compute.	More <code>num_workers</code> , local disk instead of network storage, larger batches.

LAB 22 — KNOW YOUR ACCELERATOR

1. Run the three diagnostic commands on your machine — or in Colab if you have no GPU — and write down what each layer reports.
2. Add the three-line device selection to your project and print the chosen device at startup.
3. Deliberately provoke an out-of-memory error in Colab with an oversized batch, read the message, and fix it by halving the batch.

Verify — your code names its device on startup and runs unchanged on CPU, mps and cuda.

Where to look it up

pytorch.org/get-started/locally – the current install selector; trust it over any tutorial
docs.nvidia.com/deploy/cuda-compatibility – driver and runtime compatibility

23 Renting GPUs, and not being surprised by the bill

Renting a GPU by the hour is the normal way to get serious compute, and the mechanics are simple. The part that hurts people is billing: an idle machine costs the same as a busy one, storage is billed whether or not the machine is running, and nothing stops you at a budget unless you build the stop yourself.

The workflow

1. Choose an instance by VRAM, not by name. The model must fit.
 2. Start it. Note the hourly rate and the current time.
 3. Connect with SSH or Remote-SSH. (ch 17)
 4. Clone your repository, restore from the lockfile. (ch 13)
 5. Run inside tmux so a dropped connection is harmless. (ch 18)
 6. Write results somewhere permanent — not the instance disk.
 7. Destroy the instance. Then check the dashboard says \$0/hr.

Step 7 is the one people skip. Set a phone alarm for the end of your session on your first few rentals; the habit forms faster than the bill arrives.

GOLDEN RULE 22

Nothing lives only on a rented machine. If destroying it right now would lose work, you are one dropped connection from losing that work anyway.

Sizing, roughly

JOB	VRAM YOU NEED	NOTES
Inference, 7B, quantised	6–8 GB	A modest GPU, or a good laptop
Inference, 7B, half precision	16 GB	Roughly 2 GB per billion parameters, plus overhead
Fine-tune 7B with LoRA	24 GB	The common cohort project size
Full fine-tune 7B	80 GB+	Or several GPUs and a sharding strategy
Embeddings over a large corpus	8 GB, but long-running	Cheap GPU, plenty of hours — batch size matters more than card

Rent the smallest card the job fits on and run longer. A card twice as fast for three times the price is a bad trade for coursework, and a job that does not fit at all is not a price problem.

Where the money actually goes

CHARGE	WHEN IT APPLIES	HOW TO STOP IT
Compute per hour	Whenever the instance is running, busy or idle	Destroy it. Stopping is not always enough.
Persistent storage	Continuously, even when stopped	Delete volumes you are not using
Egress / bandwidth	Downloading results or weights out	Compress; keep large intermediates in the provider's storage
Idle notebook servers	Forgotten browser tab holding a session open	Idle-shutdown settings, plus a habit of closing sessions
Spot instance interruption	Cheap, but can be taken away mid-run	Checkpoint every few minutes; treat interruption as normal

Controls worth setting on day one

- A billing alert at a number that would annoy you, and a second at a number that would hurt.
- A hard spending cap where the provider offers one.
- A prepaid balance rather than a credit card on file, where that is an option.
- Idle-shutdown enabled on every instance and notebook.
- A daily glance at the dashboard for the first fortnight, until you trust yourself.

When it goes wrong

SYMPTOM	CAUSE	FIX
A bill much larger than expected	An instance left running, or storage never deleted.	Check every region and every project; providers hide idle resources well. Then set the alerts above.
Instance destroyed with your only copy of the results	No permanent storage step.	Nothing to do now. Golden rule 22 from here on.
Job killed halfway through	Spot interruption, or a wall-clock limit.	Checkpoint and resume; keep runs short enough to survive.
Nothing available in your region	GPU scarcity.	Another region, a different card, or a queue. Plan around deadlines, not on them.
Machine is fast but the job is not	Data loading or network storage bottleneck.	Copy data to local disk first; raise <code>num_workers</code> . Chapter 22.
Cannot connect after starting	SSH key not registered with the provider, or firewall.	Chapter 17; Appendix C for blocked ports.

LAB 23 — ONE RENTED HOUR, DONE PROPERLY

1. Before starting anything, set a billing alert and write down the hourly rate you are about to accept.
2. Rent the cheapest GPU instance available, connect over SSH, and run your project's tests inside `tmux`.
3. Write one output file back to your own machine with `rsync`.
4. Destroy the instance and any volume it created. Confirm the dashboard shows nothing running.
5. Write down what the hour cost. That number is your unit of judgement for the rest of the programme.

Verify — the dashboard shows zero running resources, and your result file is on your own disk.

24 Docker, from first principles

Docker packages an application together with its entire operating environment — libraries, system packages, interpreter — into an image that runs the same way anywhere the Docker engine runs. For AI work its value is specific: it is the only reliable answer to "it works on my machine", and it is how you hand a GPU job to a machine you have never logged into.

Three words, precisely

WORD	MEANING
Image	A read-only filesystem plus a default command. Built once, shared, versioned by tag.
Container	A running instance of an image, with its own writable layer that is discarded when it is removed.
Volume	Storage that outlives containers. The only place data belongs.

A container is not a virtual machine. It shares the host kernel, starts in a fraction of a second, and cannot see your files unless you deliberately mount them. That last point causes half of all beginner confusion: the container's `/app` is not your `/app`.

The commands you will actually use

```
$ docker build -t summarise:0.1 .           # build from the Dockerfile here
$ docker run --rm -it summarise:0.1         # run, interactive, delete on exit
$ docker run --rm -v "$PWD/data:/app/data" summarise:0.1 data/input.txt
$ docker ps -a                             # what is running, and what is lying
around
$ docker logs <id>                         # its output
$ docker exec -it <id> bash                 # a shell inside a running container
$ docker stop <id> && docker rm <id>
$ docker images                            # disk usage by image
$ docker system df                          # where the gigabytes went
$ docker system prune -a                   # reclaim them. Read the warning first.
```


A Dockerfile for the project

```
FROM python:3.11-slim

ENV PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1
WORKDIR /app

# dependencies first: this layer is cached until the lockfile changes
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# then the code, which changes every commit
COPY src/ ./src/
COPY pyproject.toml .
RUN pip install --no-deps -e .

ENTRYPOINT ["summarise"]
```

Listing 24.1 — Layer order is the whole performance story: dependencies before code means editing a file rebuilds in seconds, not minutes.

```
.dockerignore
.venv/
.git/
data/
__pycache__/
*.ipynb
models/
```

Without a `.dockerignore`, `docker build` sends your entire project — environment, git history, datasets — to the daemon before it starts. This is the difference between a two-second build context and a two-minute one.

GOLDEN RULE 23

Never bake a secret into an image. Layers are readable by anyone who has the image, including the ones you thought you deleted.

Do you need Docker at all?

Not for coursework on your own machine — a lockfile does that job with less ceremony. You need it when you are deploying, when you must reproduce someone's exact CUDA environment, when a project needs system libraries that fight your OS, or when a team must share one environment exactly. It also requires admin rights to install, so for part of any cohort the honest answer is: read this chapter, use Colab, and return to it when you have a machine you control.

When it goes wrong

SYMPTOM	CAUSE	FIX
Cannot connect to the Docker daemon	Docker Desktop or the service is not running.	Start it. On Linux, <code>sudo systemctl start docker</code> and add yourself to the <code>docker</code> group.
Build is slow every single time	Layer order, or no <code>.dockerignore</code> .	Dependencies before code; add the ignore file.
Container exits immediately	Its command finished, or it crashed.	<code>docker logs</code> . A container lives only as long as its process.
no such file or directory for a file you can see	It was never copied in, or is outside the mount.	<code>docker run --rm -it image bash</code> and look. Mount with <code>-v</code> .
Written files vanish on exit	They were in the container's writable layer.	Write to a mounted volume.
Files created in the container are owned by root	Default user is root, and the mount is shared.	<code>--user "\$(id -u):\$(id -g)"</code> , or create a non-root user in the image.
exec format error	Image built for a different architecture — common on Apple Silicon.	Build for your platform, or <code>--platform linux/amd64</code> and accept emulation.
Disk full	Old images, stopped containers, build cache.	<code>docker system df</code> , then <code>prune</code> . Chapter 26.

LAB 24 — SUMMARISE-CLI, STEP 13

1. Write the Dockerfile and `.dockerignore` from this chapter. Build the image and note how long it takes.
2. Change one line of your code and rebuild. If it took more than a few seconds, your layer order is wrong.
3. Run the container with your data directory mounted and your API key passed in as an environment variable — not baked in.
4. Shell into the running container and look around: find your code, your Python, and confirm your `.env` is *not* there.

Verify — `docker run --rm -v "$PWD/data:/app/data" -e OPENAI_API_KEY summarise:0.1 data/input.txt` produces a summary.

PART IV – COMPUTE

25 Docker in practice: images, volumes, GPU containers

The previous chapter got a container running. This one covers the three things that make containers usable for real work: keeping images small, getting data in and out safely, and reaching the GPU.

Smaller images, in four moves

- **Start from `-slim`.** `python:3.11-slim` is around 150 MB against a full image's 1 GB. Use the full one only when you need build tools.
- **Combine and clean in one `RUN`.** Each `RUN` is a layer; deleting a file in a later layer does not shrink the earlier one. `apt-get install ... && rm -rf /var/lib/apt/lists/*` in a single instruction.
- **Multi-stage builds.** Compile in one stage, copy only the result into a clean final stage. This is how a 3 GB build becomes a 300 MB image.
- **Do not copy what you do not need.** The `.dockerignore` from chapter 24 is most of this.

```
FROM python:3.11 AS build
WORKDIR /app
COPY requirements.txt .
RUN pip wheel --wheel-dir /wheels -r requirements.txt

FROM python:3.11-slim
WORKDIR /app
COPY --from=build /wheels /wheels
RUN pip install --no-index --find-links=/wheels /wheels/*.whl && rm -rf /wheels
COPY src/ ./src/
ENTRYPOINT ["python", "-m", "summarise_cli"]
```

Listing 25.1 — Two stages: the compiler and its debris stay in the first, which is thrown away.

Getting data in and out

FLAG	WHAT IT DOES
<code>-v "\$PWD/data:/app/data"</code>	Bind-mount a host directory. Your files, visible live inside the container.
<code>-v modelcache:/root/.cache</code>	Named volume — persists between containers. Use for model and package caches.
<code>--env-file .env</code>	Pass secrets at run time. Never <code>COPY .env</code> .
<code>-e OPENAI_API_KEY</code>	Pass one variable through from your shell without naming its value.
<code>-p 8888:8888</code>	Publish a port — reach a container's Jupyter or API from your browser.
<code>--user "\$(id -u):\$(id -g)"</code>	Write files as you, not root.
<code>--rm</code>	Delete the container on exit. Use it by default.

GPU containers

```
$ # Linux host: install the NVIDIA Container Toolkit once, then
$ docker run --rm --gpus all nvidia/cuda:12.1.0-base-ubuntu22.04 nvidia-smi

$ # your own image, with the GPU and a persistent model cache
$ docker run --rm --gpus all \
  -v "$PWD/data:/app/data" \
  -v hfcache:/root/.cache/huggingface \
  --env-file .env \
  summarise:0.1 data/input.txt
```

The rule from chapter 22 holds: the driver belongs to the host, the container brings the runtime. On Windows this works through WSL2 with the Windows driver installed. On Apple Silicon it does not work at all — there is no GPU passthrough to Linux containers, so Mac users do GPU work natively with `mps` or on a remote machine.

docker compose, for when one container is not enough

```
services:
  app:
    build: .
    env_file: .env
    volumes:
      - ./data:/app/data
      - hfcache:/root/.cache/huggingface
    deploy:
      resources:
        reservations:
          devices: [{ capabilities: [gpu] }]
volumes:
  hfcache:
```

`docker compose up` then replaces a long `docker run` line, and the file documents the run configuration in your repository. That is reason enough to use it even for a single service.

GOLDEN RULE 24

Data in volumes, secrets in the environment, code in the image. Mixing those three up is how images become both enormous and unsafe.

When it goes wrong

SYMPTOM	CAUSE	FIX
could not select device driver ... capability: [[gpu]]	NVIDIA Container Toolkit not installed.	Install it on the host; restart Docker; retry the <code>nvidia-smi</code> test above.
<code>--gpus all</code> works, torch still sees no GPU	CPU wheel inside the image.	Install the CUDA wheel in the Dockerfile, or use a CUDA base image. Chapter 22.
Image is 8 GB	Full base image, build tools, cached wheels, copied data.	Slim base, multi-stage, <code>--no-cache-dir</code> , <code>.dockerignore</code> .
Models re-download on every run	The cache lives in the container's writable layer.	Mount a named volume at the cache path. Chapter 26.
Bind mount is empty inside the container	Relative path, or a path Docker Desktop is not allowed to share.	Absolute paths (<code>"\$PWD/data"</code>); check Docker Desktop's file sharing settings.
Very slow file access on Mac or Windows	Bind mounts cross a virtualisation boundary.	Keep hot data in a named volume; mount only what you need.
Container cannot reach the network	Corporate DNS or proxy.	Appendix C — Docker needs its own proxy configuration.
<code>.env</code> ended up in the image	<code>COPY . .</code> without a <code>.dockerignore</code> .	Rotate the key, fix the ignore file, rebuild, delete the old image and any pushed tags.

LAB 25 — SUMMARISE-CLI, STEP 14

1. Convert your Dockerfile to multi-stage and compare `docker images` before and after.
2. Write a `compose.yaml` that mounts your data, loads `.env`, and keeps a named cache volume.
3. Run the same job twice and confirm the second run reuses the cache.
4. If you have an NVIDIA GPU, run the `nvidia-smi` container test and then your own image with `--gpus all`.

Verify — `docker compose up` runs your tool, the image is under about 500 MB, and no secret is inside it.

26 Model caches and disk hygiene

Within a month of starting this work, someone in every cohort runs out of disk space, and the cause is always the same: caches that were designed to be invisible. Nothing warned them, because nothing is wrong — the tools are behaving as intended. This chapter is where the space went and how to get it back safely.

Where the space goes

WHAT	DEFAULT LOCATION	TYPICAL SIZE
Hugging Face models	<code>~/.cache/huggingface/hub</code>	Tens to hundreds of GB
pip cache	<code>~/.cache/pip</code> · <code>%LOCALAPPDATA%\pip</code>	1–10 GB
uv cache	<code>~/.cache/uv</code>	1–10 GB
conda packages	<code>~/miniforge3/pkgs</code>	5–50 GB
Docker images and build cache	Docker's own storage	10–100 GB
PyTorch hub / torch cache	<code>~/.cache/torch</code>	Several GB
Ollama models	<code>~/.ollama/models</code>	Tens of GB
Old virtual environments	<code>.venv</code> in every project you ever started	0.5–5 GB each

Find it, then clear it

```
$ du -sh ~/.cache/* ~/.ollama 2>/dev/null | sort -h | tail -12
$ docker system df
$ du -sh ~/projects/*/venv | sort -h | tail

$ # safe, tool-aware cleanup — each of these only removes cache
$ huggingface-cli delete-cache      # interactive: choose revisions to drop
$ pip cache purge
$ uv cache clean
$ conda clean --all
$ docker system prune -a --volumes  # read the warning: this deletes unused
volumes too
$ ollama list && ollama rm <model>
```

Listing 26.1 — Prefer each tool's own cleanup command over deleting directories by hand; they know what is safe to remove.

GOLDEN RULE 25

A cache is never data. If deleting something would lose work rather than time, it is not a cache and does not belong in one.

Move the caches instead of fighting them

If your system disk is small and you have a larger external or secondary drive, point the caches at it once and forget about it. Set these in your shell profile so every project inherits them.

```
export HF_HOME="/Volumes/big/hf"           # Hugging Face: models, datasets, tokens
export UV_CACHE_DIR="/Volumes/big/uv"
export PIP_CACHE_DIR="/Volumes/big/pip"
export TORCH_HOME="/Volumes/big/torch"
export OLLAMA_MODELS="/Volumes/big/ollama"
```

In Docker, do the equivalent with a named volume mounted at the cache path (chapter 25) — that also stops every container rebuild from re-downloading a 14 GB model.

Habits that prevent the problem

- Check free space before downloading weights. A model page states its size; a 70B model in half precision is around 140 GB.
- Delete `.venv` from projects you are not working on this month. With a lockfile it costs seconds to restore (chapter 13).
- Prefer quantised weights when you only need inference — often a quarter of the size at a small quality cost.
- Keep datasets out of your home directory and out of git. One canonical copy, referenced by path.
- Run `docker system df` monthly. It is always more than you expect.

When it goes wrong

SYMPTOM	CAUSE	FIX
No space left on device mid-download	Disk full; the partial file is still there.	Clear a cache from Listing 26.1, then re-download — resumable in most tools.
Deleted a model, disk still full	Hugging Face keeps blobs and symlinked revisions.	<code>huggingface-cli delete-cache</code> , which understands the layout.
Machine slows to a crawl near a full disk	No room for swap or temporary files.	Free 10% of the disk before anything else. Then diagnose.
Model re-downloads every run	Cache path not persisted — a container, or a changed <code>HF_HOME</code> .	Mount a volume; set the variable in your profile, not per-command.
<code>docker system prune</code> deleted something you wanted	<code>-a --volumes</code> removes unused volumes too.	Nothing to recover. Use <code>prune</code> without <code>--volumes</code> unless you are sure.
Cache on an external drive breaks when unplugged	The path no longer exists.	Expected. Unset the variable, or keep a small local cache for daily work.

LAB 26 — RECLAIM TEN GIGABYTES

1. Record your free disk space. Then run the three `du` commands from Listing 26.1 and rank the top five consumers.
2. Clear two caches with their own commands. Record the space recovered.
3. Set `HF_HOME` deliberately — even if only to confirm where it already points — and note it in your dotfiles.
4. Delete `.venv` from one dormant project and restore it from the lockfile to prove the round trip works.

Verify — you can name the four largest caches on your machine and the command that safely empties each.

PART IV — WHAT AGES FASTEST

Every specific number here: CUDA suffixes, VRAM requirements, model sizes, Colab's limits and hourly rates. Check the install selector on pytorch.org and your provider's own pricing page rather than this book. The four-layer model in chapter 22, the ephemeral-machine discipline in chapters 21 and 23, and the image/container/volume distinction in chapter 24 are structural and will hold.

27

API keys and secrets: storage, rotation, and what to do after a leak

An API key is a password that costs money. A leaked one is not embarrassing; it is a bill and, sometimes, someone else's data. This is the chapter to read before you create your first key, and the one to reread the day you leak one — which most people do eventually, and which is survivable if you act in the first ten minutes.

The rule, and the three places a secret may live

A secret never appears in source code, in a notebook cell, in a commit, in a screenshot, in a chat message, or in a container image. It lives in exactly one of three places: an environment variable loaded from an ignored file, your operating system's keychain, or a platform's own secret store.

WHERE	GOOD FOR	HOW
<code>.env</code> file, git-ignored	Local development, the common case	<code>python-dotenv</code> ; <code>chmod 600 .env</code>
OS keychain	Keys you use across many projects	<code>keyring</code> package; macOS Keychain, Windows Credential Manager, libsecret
Colab Secrets	Notebook work	Key icon in the sidebar; <code>userdata.get()</code> (chapter 21)
GitHub Actions secrets	CI	Repository settings; referenced as <code>\${{ secrets.NAME }}</code> (chapter 29)
Cloud secret manager	Anything deployed	AWS Secrets Manager, GCP Secret Manager, or your provider's equivalent

The local pattern, complete

```
$ cat .gitignore
.env
.env.*
!.env.example

$ cat .env.example      # committed: names only, no values
OPENAI_API_KEY=
ANTHROPIC_API_KEY=
GOOGLE_API_KEY=
HF_TOKEN=

$ cat .env              # never committed
OPENAI_API_KEY=sk-...
$ chmod 600 .env
```

```
import os
from dotenv import load_dotenv

load_dotenv()                # reads .env if present
key = os.environ.get("OPENAI_API_KEY")
if not key:
    raise SystemExit(
        "OPENAI_API_KEY is not set. Copy .env.example to .env and fill it in."
    )
```

Listing 27.1 — The `.env.example` file is how a newcomer knows which keys the project needs without you sending them any. The explicit error message is worth more than a stack trace six frames deep.

GOLDEN RULE 26

Add `.env` to `.gitignore` in the first commit of every project, before the file exists. Retrofitting it is a rewrite of history; doing it first is one line.

Per-provider notes

PROVIDER	VARIABLE	NOTES
OpenAI	<code>OPENAI_API_KEY</code>	Project-scoped keys; set a per-project spend limit in the dashboard
Anthropic	<code>ANTHROPIC_API_KEY</code>	Workspaces separate coursework from anything real
Google Gemini / Vertex	<code>GOOGLE_API_KEY</code> or a service-account JSON	Vertex uses <code>GOOGLE_APPLICATION_CREDENTIALS</code> pointing at a file — that file is a secret too
Hugging Face	<code>HF_TOKEN</code>	Read-only tokens for downloading; write tokens only when pushing
Local models (Ollama)	none	No key, no cost, no leak. A good default for experiments.
AWS / GCP	<code>AWS_PROFILE</code> , <code>GOOGLE_APPLICATION_CREDENTIALS</code>	Never long-lived root keys; use profiles and short-lived credentials

Hygiene that costs nothing

- **One key per person per project.** Shared keys cannot be rotated without breaking everyone, and cannot tell you who spent the money.
- **Least privilege.** Read-only where reading is all you do.
- **Spend limits at the provider.** The only control that works while you are asleep.
- **Rotate on a schedule** — quarterly is fine for coursework — and immediately on any doubt.
- **Never print a key.** Not to check it. Log the last four characters if you must confirm which key is loaded.
- **Enable secret scanning** on your repositories, and keep `detect-private-key` in pre-commit (chapter 19).

You leaked a key. Do this, in this order.

1. **Revoke it at the provider.** Now, before anything else. A revoked key cannot be misused, and no amount of git surgery achieves that.
2. **Issue a new one** and put it in your `.env`.
3. **Check usage** in the provider's dashboard for calls you did not make.
4. **Remove it from history** with `git filter-repo` (or the provider's guidance), then force-push and tell collaborators to re-clone. This is cleanup, not remediation — assume the key was captured the moment it was pushed.

5. **Fix the cause.** The ignore rule that was missing, the notebook cell that printed it, the image that copied it.
6. **Tell whoever needs to know.** On a course, your instructor. At work, your security team. Quietly hoping is how a small incident becomes a large one.

When it goes wrong

SYMPTOM	CAUSE	FIX
GitHub emails you about an exposed secret	Secret scanning found a pushed key.	The six steps above. Start with revocation.
Unexpected charges on an API account	Leaked key, or a runaway loop of your own.	Revoke, read the usage log, set a spend limit. Chapter 28.
401 Unauthorized / invalid_api_key	Not loaded, wrong variable name, revoked, or whitespace in the value.	<pre>python -c "import os;k=os.environ.get('OPENAI_API_KEY');print(bool(k), k and len(k))"</pre>
Works in the terminal, fails in the notebook	The kernel did not inherit your shell's variables.	<code>load_dotenv()</code> in the notebook, or launch Jupyter from the activated shell.
<code>.env</code> committed three months ago	Missing ignore rule.	Assume public. Rotate everything in it, then clean history.
Key visible in a screenshot you posted	It was on screen.	Rotate. Screenshots are indexed and archived.
Container has your <code>.env</code> inside	<pre>COPY . . without .dockerignore .</pre>	Rotate, fix the ignore file, rebuild, delete pushed tags. Chapter 25.

LAB 27 – SUMMARISE-CLI, STEP 15

1. Create the `.env` / `.env.example` pair, ignore the first, commit the second, and `chmod 600` the real one.
2. Load the key with `python-dotenv` and add the explicit error from Listing 27.1. Test it by unsetting the variable.
3. Set a spend limit on the provider you are using. Write the number in your README.
4. Rotate your key once now, while nothing depends on it, so the procedure is familiar before it is urgent.
5. Run `git log -p | grep -i "sk-"` over your repository and confirm it finds nothing.

Verify — the tool runs from `.env`, fails with a clear message when the key is missing, and no key exists anywhere in git history.

28 Calling model APIs: rate limits, retries, token cost

A model API is an HTTP call that is slow, occasionally fails, and costs money in proportion to the text going in and out. Every robust piece of AI engineering handles those three facts explicitly, and code that ignores them works beautifully in a demo and falls over the first time it meets a hundred documents.

Tokens, and therefore money

Models bill per token — roughly three-quarters of a word in English — and input and output are usually priced differently, with output dearer. Two consequences shape how you write code: long prompts repeated in a loop are the usual cause of a surprising bill, and asking for shorter outputs is often the cheapest optimisation available.

```
$ python - <<'PY'
import tiktoken
enc = tiktoken.get_encoding("cl100k_base")
text = open("data/input.txt").read()
n = len(enc.encode(text))
print(f"{n} tokens; at $0.15 per 1M input tokens that is ${n/1e6*0.15:.4f}")
PY
```

Listing 28.1 — Count before you loop. Multiply by the number of documents *and* the number of times you will rerun while debugging, which is the term everyone forgets.

The error codes, and what each one means you should do

STATUS	MEANING	RESPONSE
400	Malformed request; often too many tokens	Do not retry. Fix the request; shorten the input.
401	Bad or missing key	Do not retry. Chapter 27.
403	Not permitted — model, region or scope	Do not retry. Check access to that model.
404	Unknown model name	Do not retry. Usually a typo or a deprecated name.
429	Rate limited, or out of quota	Retry with exponential backoff and jitter. Honour Retry-After .
500 / 502 / 503	Provider-side failure	Retry with backoff, a bounded number of times.
Timeout	Slow generation or a network stall	Set an explicit timeout; retry idempotent calls only.

Retries, written once

```
import os, random, time
from openai import OpenAI, RateLimitError, APIError

client = OpenAI(timeout=60, max_retries=0)      # own the retry policy explicitly

def complete(prompt: str, attempts: int = 5) -> str:
    for i in range(attempts):
        try:
            r = client.chat.completions.create(
                model="gpt-4o-mini",
                messages=[{"role": "user", "content": prompt}],
                max_tokens=300,                # a cost ceiling, not a suggestion
            )
            return r.choices[0].message.content
        except (RateLimitError, APIError) as e:
            if i == attempts - 1:
                raise
            wait = min(2 ** i, 30) + random.uniform(0, 1)  # backoff + jitter
            print(f"{type(e).__name__}: retrying in {wait:.1f}s")
            time.sleep(wait)
```

Listing 28.2 — Exponential backoff with jitter. The jitter matters: without it, every parallel worker retries in lockstep and rate-limits itself again.

GOLDEN RULE 27

Retry on 429 and 5xx. Never retry on 400, 401, 403 or 404 — the same request will fail identically, and you will have paid for the privilege.

Six habits that keep the bill small

- **Develop against ten records, not ten thousand.** Add a `--limit` flag and use it by default.
- **Cache responses on disk during development.** Keyed on a hash of the prompt, so a rerun costs nothing.
- **Cap output.** `max_tokens` is the only hard ceiling on the expensive half of the bill.
- **Use the smallest model that passes your evaluation.** The difference between tiers is often ten to twenty times the price.
- **Log tokens and estimated cost per call.** A running total in your own logs makes waste visible the same day.
- **Try a local model first** for anything you will run thousands of times while iterating (chapter 27's note on Ollama).

When it goes wrong

SYMPTOM	CAUSE	FIX
Constant 429s from the first call	New account with low limits, or the wrong tier.	Backoff; batch; request a limit increase. Do not parallelise harder.
A loop spent a large amount of money quickly	Retries inside retries, or no <code>max_tokens</code> .	One retry layer only; cap output; set a provider spend limit. Chapter 27.
<code>context_length_exceeded</code>	Prompt plus expected output exceeds the model's window.	Chunk the input; summarise hierarchically; count tokens first.
Script hangs indefinitely	No timeout configured.	Always pass a timeout. Sixty seconds is a reasonable default.
Works on ten records, fails on a thousand	Rate limits, memory, or no resumability.	Backoff, batching, and a checkpoint so a failure resumes rather than restarts.
Different results every run	Sampling is non-deterministic by design.	Lower the temperature and set a seed where supported; expect variation regardless.
Cost per record much higher than estimated	Output tokens, or a system prompt repeated on every call.	Log both counts per call; shorten the repeated part.

LAB 28 – SUMMARISE-CLI, STEP 16

1. Make the real API call. Print the summary, the token counts, and the estimated cost of that one call.
2. Add the retry helper from Listing 28.2, and a `--max-tokens` option that defaults to something modest.
3. Add a disk cache keyed on the hash of the input plus the options. Run twice; confirm the second run makes no call.
4. Run the tool over ten files, logging cumulative cost. Extrapolate to a thousand and decide whether you would spend it.

Verify — the tool survives a simulated 429 (raise one in a test), caches repeats, and reports its own cost.

Where to look it up

Your provider's `rate-limit` and `pricing` pages – both change; never hard-code a price from a book

29 Continuous integration with GitHub Actions

Continuous integration is a machine that clones your repository from scratch and runs your checks on every push. Its real value for a learner is not the checks — it is that the clean machine cannot see anything you forgot to commit. If it passes there, it will pass on your classmate's laptop. It is chapter 13's promise, tested automatically, forever.

A workflow that earns its keep

```
.github/workflows/ci.yml
name: ci
on:
  push: { branches: [main] }
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: astral-sh/setup-uv@v3
        with:
          enable-cache: true

      - run: uv python install 3.11
      - run: uv sync --frozen           # fails if the lockfile is stale — the point
      - run: uv run ruff check .
      - run: uv run pytest -q

      - name: smoke test the CLI
        env:
          OPENAI_API_KEY: ${ secrets.OPENAI_API_KEY }
        run: uv run summarise data/sample.txt --limit 1
```

Listing 29.1 — Four checks. `--frozen` is the most valuable line in the file: it fails when someone adds a dependency without re-locking.

GOLDEN RULE 28

CI runs on a machine that has never met you. Whatever it cannot find, you forgot to commit — and your teammate would have hit the same wall tomorrow.

Secrets in CI

Repository settings → Secrets and variables → Actions. They are write-only: you can set them and never read them back, and they are masked in logs. Two cautions. A pull request from a fork does not get your secrets, by design — so a test that requires a key will not run on outside contributions. And a smoke test that calls a paid API runs on every push, so keep it to one tiny call, or guard it behind a manual trigger.

What to check, in order of value

CHECK	COST TO ADD	WHAT IT CATCHES
Install from the lockfile	Two lines	Uncommitted dependencies; stale lockfiles. The single highest-value check.
Lint	One line	Unused imports, undefined names, obvious mistakes
Tests	One line, plus tests	Regressions, and the fact that you have tests at all
Smoke test the entry point	Three lines	The failure users actually see: it does not start
Build the Docker image	Two lines	Dockerfile drift after a dependency change
Matrix over Python versions	Three lines	Version-specific breakage — only if you support several

Keeping it fast and cheap

Cache the package manager's directory (`enable-cache` above, or `actions/cache`). Run jobs only where they matter — `paths-ignore` for documentation-only changes. Never train a model in CI: minutes on a hosted runner are metered, GPU runners are expensive, and a job that takes twenty minutes is a job nobody waits for. If a check is slower than the patience of the person waiting it will be ignored, and an ignored check is worse than none.

When it goes wrong

SYMPTOM	CAUSE	FIX
Passes locally, fails in CI	Something uncommitted, or an environment variable only you have.	Read the log from the top. This is CI working correctly.
<code>uv sync --frozen</code> fails	Lockfile does not match the manifest.	Re-lock locally and commit the lockfile.
<code>ModuleNotFoundError</code> in CI only	A dependency you installed by hand and never declared.	Add it properly. Chapter 13.
Secret is empty in the job	Not set, misspelled, or a fork pull request.	Check the name; skip key-dependent steps when the secret is absent.
Workflow does not trigger	Wrong branch or <code>on:</code> filter; file not in <code>.github/workflows/</code> .	Check the path and the branch. YAML indentation errors also silence it.
Every run takes ten minutes	No caching; reinstalling everything each time.	Enable the cache; split slow jobs; skip on docs-only changes.
Test that calls an API fails intermittently	Rate limits or network flakiness.	Mock the API in unit tests; keep one real smoke call. Chapter 28.

LAB 29 — SUMMARISE-CLI, FINAL STEP

1. Add the workflow from Listing 29.1. Push it and watch it run.
2. Break it deliberately: add an import without declaring the dependency, push, and read the failure. Then fix it properly.
3. Add your API key as a repository secret and make the smoke test run — one call, one document.
4. Add a status badge to the README. Then read your own README as if you had never seen the project: could you set it up in two commands?
5. Finally, in a completely clean directory: clone, `make setup`, `make test`, `make run`. If all four work, this manual has done its job.

Verify — a green check on `main`, and a fresh clone that runs your tool in two commands.

PART V – WHAT AGES FASTEST

Action version tags, SDK class names, model names and every price in chapter 28. The three-place rule for secrets, the leak procedure, the retry policy by status code and the value of installing from a lockfile on a clean machine will all still be right long after the code samples need editing.

APPENDIX

A Pre-flight checklists, one per platform

Work down your platform's list. Every line is a command whose output you can check, not a feeling. When all of them pass you are ready for the programme; if one fails, the chapter reference tells you where to go. Tick them in pencil — you will do this again on your next machine.

MACOS · ZSH

- ☐ `uname -m` — know if you are arm64 or x86_64 (02)
- ☐ `xcode-select --install` — command line tools present
- ☐ `echo $SHELL` → zsh, and you know where `~/.zshrc` is (03)
- ☐ `python3.11 -V` works (05)
- ☐ `which -a python3` — you can name each one (04)
- ☐ `~/.local/bin` on PATH (04)
- ☐ A `.venv` created and activated (06)
- ☐ `git -v`, and `user.email` configured (15)
- ☐ `ssh -T git@github.com` greets you (17)
- ☐ VS Code + `code` command, interpreter selected (14)
- ☐ `torch.backends.mps.is_available()` is True (22)
- ☐ `.env` ignored, `chmod 600`, key loads (27)
- ☐ ≥ 50 GB free, and you know where `HF_HOME` points (26)

UBUNTU / DEBIAN · BASH

- ☐ `lsb_release -a` — know your version (02)
- ☐ `python3.11 -V` and `python3.11-venv` installed (05)
- ☐ `build-essential` present, or you know you have no compiler
- ☐ `~/.local/bin` on PATH in `~/.bashrc` (04)
- ☐ A `.venv` created and activated (06)
- ☐ `git -v`, identity configured (15)
- ☐ SSH key generated and registered (17)
- ☐ `tmux -V` — installed, and you can detach (18)
- ☐ `nvidia-smi` — driver version noted, or "no GPU" accepted (22)
- ☐ `torch.cuda.is_available()` matches the line above (22)
- ☐ `docker run hello-world` without `sudo` (24)
- ☐ `.env` ignored, `chmod 600`, key loads (27)
- ☐ `df -h /` — ≥ 50 GB free (26)

WINDOWS + WSL2 · BASH

- `wsl -l -v` shows version 2 (02)
- Projects live in `~`, not `/mnt/c` (02)
- `python3.11 -V` inside WSL2 (05)
- A `.venv` created and activated (06)
- `git -v`, identity, `core.autocrlf input` (15)
- SSH key in WSL2, registered with GitHub (17)
- `code .` from WSL2 opens VS Code, extensions installed in the WSL2 context (14)
- NVIDIA driver installed on *Windows*, not in Linux (22)
- `nvidia-smi` works inside WSL2 (22)
- Docker Desktop with WSL2 integration on (24)
- `.env` ignored, `chmod 600`, key loads (27)
- Disk space checked on both sides (26)

WINDOWS NATIVE · POWERSHELL

- `$PSVersionTable` — 7.x preferred (03)
- Store alias for `python` disabled (05)
- `py --list` shows 3.11 (05)
- `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` (06)
- `.\.venv\Scripts\Activate.ps1` works (06)
- `where.exe python` → inside `.venv` (04)
- `git -v`, identity, `core.autocrlf input` (15)
- SSH key generated, agent service running (17)
- VS Code interpreter points at `.venv\Scripts\python.exe` (14)
- `nvidia-smi` and the matching torch wheel, or CPU accepted (22)
- A task runner that is not `make` — `just` or scripts (19)
- `.env` ignored, key loads (27)

CLOUD ONLY · COLAB AND RENTED MACHINES

- | | |
|---|--|
| ☐ GitHub account, repository created, SSH or CLI auth working (16, 17) | ☐ Repository clones and installs from the lockfile in one cell (13, 21) |
| ☐ Notebook opens with a GPU runtime; <code>nvidia-smi</code> run and read (21) | ☐ You have destroyed a rented instance and confirmed \$0/hr (23) |
| ☐ Keys in the Secrets panel, never in a cell (21, 27) | ☐ Billing alert set at a number that would annoy you (23) |
| ☐ Drive mounted; you have saved and recovered one file (21) | ☐ Provider spend limit set on every API key (27, 28) |

GOLDEN RULE 29

Run the checklist for your platform before every module, not just the first. Ten minutes of checking prevents the class of problem that eats an afternoon.

APPENDIX

B The day-one setup script, annotated

Five scripts ship alongside this manual — macOS, Ubuntu, WSL2, PowerShell and a Colab cell. They are all the same script in different dialects, they all install into your home directory without administrator rights, and they are all safe to run twice: each step checks whether the work is already done and says so rather than doing it again.

What it does, in order

STEP	ACTION	WHY
1	Report OS, architecture, shell, free disk	Chapter 02's inventory, so the log tells you what machine this was
2	Ensure <code>~/local/bin</code> exists and is on PATH	Every user-level install lands there (chapter 04)
3	Install <code>uv</code> if absent	Gives Python 3.11 without admin rights or a compiler (chapter 09)
4	<code>uv python install 3.11</code>	The pinned interpreter (chapter 05)
5	Check for git; configure identity if unset	Refuses to guess your name and email — it prompts (chapter 15)
6	Generate an ed25519 SSH key if none exists	Prints the public half for you to paste into GitHub (chapter 17)
7	Create the project skeleton and <code>.venv</code>	Lab 6's starting point, ready to go
8	Write <code>.gitignore</code> , <code>.env.example</code> , <code>.python-version</code>	The three files that must exist before the first commit (chapters 15, 27)
9	Print a verification block	Every command from your platform's checklist, with its answer

What it deliberately does not do

It does not install Docker, touch a driver, install conda, or modify anything outside your home directory — all four need administrator rights or carry real risk, and a script is the wrong place for a decision you should make with your eyes open. It also never writes a secret: step 8 creates `.env.example` with empty values and leaves `.env` to you.

The core of the bash version

```
#!/usr/bin/env bash
set -euo pipefail          # stop on the first error, not the last

say() { printf '\n\033[1m== %s\033[0m\n' "$1"; }
have() { command -v "$1" >/dev/null 2>&1; }

say "1. machine"
printf 'os=%s arch=%s shell=%s\n' "$(uname -s)" "$(uname -m)" "$SHELL"
df -h "$HOME" | tail -1

say "2. ~/.local/bin on PATH"
mkdir -p "$HOME/.local/bin"
case ":$PATH:" in
  *"$HOME/.local/bin:*)" echo "already on PATH – skipping" ;;
  *) echo 'export PATH="$HOME/.local/bin:$PATH"' >> "$RC_FILE"
    export PATH="$HOME/.local/bin:$PATH"
    echo "added to $RC_FILE" ;;
esac

say "3. uv"
if have uv; then echo "present: $(uv --version) – skipping"
else curl -LsSf https://astral.sh/uv/install.sh | sh; fi

say "4. python 3.11"
uv python install 3.11
uv python find 3.11
```

Listing B.1 — The shape of every step: check, report, act only if needed. `set -euo pipefail` is what makes a failure visible instead of silent.

Running it

```
$ # macOS
curl -O <url>/setup-macos.sh && bash setup-macos.sh

$ # Ubuntu / WSL2
bash setup-ubuntu.sh      # or setup-wsl2.sh

PS> # Windows PowerShell
Set-ExecutionPolicy -Scope Process Bypass; .\setup-windows.ps1

# Colab: paste setup-colab.py into the first cell
```

Read the script before you run it. It is sixty lines, it is commented, and reading a setup script before executing it is a habit worth more than the twenty minutes this one saves you (chapter 03).

GOLDEN RULE 30

A setup script you can rerun safely is documentation that cannot go stale. A setup script you are afraid to rerun is a liability.

When it goes wrong

SYMPTOM	CAUSE	FIX
<code>Permission denied</code> running it	Not executable.	<code>bash script.sh</code> works regardless of the execute bit.
Stops at step 3 with a TLS error	Corporate proxy inspecting traffic.	Appendix C, then rerun — it resumes where it stopped.
<code>uv: command not found</code> immediately after step 3	PATH changed in the script's shell, not yours.	Open a new terminal, or <code>source</code> your rc file. The script says so at the end.
PowerShell refuses to run it	Execution policy.	The <code>-Scope Process Bypass</code> line above; it reverts when the window closes.
Second run reports everything as skipped	It is idempotent.	Working as intended.
Step 6 says a key already exists	You have one.	Correct behaviour — it will never overwrite a key.

APPENDIX

C Networks, proxies and firewalls

This appendix exists because network problems disguise themselves as tool problems. A corporate proxy makes pip look broken, git look misconfigured and Docker look unreliable, and the error messages point at the wrong layer every time. If a download, clone or pull fails for a reason you cannot explain, come here before you reinstall anything.

Is it the network? Four tests

```
$ curl -sSI https://pypi.org/simple/ | head -1      # plain HTTPS reachable?
$ curl -sSI https://github.com | head -1
$ nc -vz ssh.github.com 443                        # is 443 open?
$ nc -vz github.com 22                             # is 22 blocked?
$ env | grep -i proxy                               # are proxy variables already
set?
```

HTTPS works but SSH times out: port 22 is blocked. Everything fails with a certificate error: TLS inspection. Some hosts work and others do not: an allowlist. All four fail: you are not on the network you think you are.

The five failures, and their fixes

FAILURE	WHAT YOU SEE	FIX
TLS inspection	SSL: CERTIFICATE_VERIFY_FAILED, self-signed certificate in certificate chain	Get your organisation's root certificate and point the tools at it (below). Do not disable verification.
Proxy not configured	Timeouts, ProxyError, hangs at 0%	Set HTTP_PROXY / HTTPS_PROXY / NO_PROXY
Port 22 blocked	ssh: connect to host github.com port 22: Connection timed out	SSH over 443 (chapter 17's config block)
Internal mirror required	Packages exist but cannot be found	index-url in pip.conf; conda channels; ask which mirror
Docker Hub blocked or rate-limited	toomanyrequests, or pulls that never start	Authenticate, use an internal registry, or build from a mirrored base image

Configuring every tool, in one place

```
# shell profile – most tools read these
export HTTP_PROXY="http://proxy.company.com:8080"
export HTTPS_PROXY="$HTTP_PROXY"
export NO_PROXY="localhost,127.0.0.1,.company.com"

# corporate root certificate, once, for everything that respects it
export REQUESTS_CA_BUNDLE="$HOME/certs/company-root.pem"
export SSL_CERT_FILE="$REQUESTS_CA_BUNDLE"
export CURL_CA_BUNDLE="$REQUESTS_CA_BUNDLE"

# pip – ~/.config/pip/pip.conf, or %APPDATA%\pip\pip.ini
[global]
proxy = http://proxy.company.com:8080
cert = /Users/you/certs/company-root.pem
index-url = https://mirror.company.com/simple

# git
git config --global http.proxy "$HTTP_PROXY"
git config --global http.sslCAInfo "$HOME/certs/company-root.pem"

# conda – ~/.condarc
proxy_servers:
  https: http://proxy.company.com:8080
ssl_verify: /Users/you/certs/company-root.pem

# uv
export UV_NATIVE_TLS=1          # use the OS trust store, which has your root cert#
# Docker daemon proxy – ~/.docker/config.json
{ "proxies": { "default": {
  "httpProxy": "http://proxy.company.com:8080",
  "httpsProxy": "http://proxy.company.com:8080",
  "noProxy": "localhost,127.0.0.1" } } }
```

Listing C.1 — Ask your IT team for the proxy address and the root certificate file. Both are ordinary, unremarkable requests.

GOLDEN RULE 31

Never fix a certificate error by turning verification off. `--trusted-host`, `ssl_verify: false` and `GIT_SSL_NO_VERIFY` trade a working install for a machine that cannot tell a real server from a fake one. Install the certificate.

Offline and metered connections

Where bandwidth is expensive or absent, download once and install many times. `pip download -r requirements.txt -d wheels/` on a good connection, then `pip install --no-index --find-links wheels/ -r requirements.txt` anywhere. Hugging Face weights can be fetched once and shared through a common `HF_HOME` (chapter 26), and `HF_HUB_OFFLINE=1` stops libraries from silently reaching for the network. Warn a cohort before any step that pulls multiple gigabytes; on a metered connection that is a real cost, not an inconvenience.

When it goes wrong

SYMPTOM	CAUSE	FIX
Works on home wifi, fails at the office	Proxy and TLS inspection only on the corporate network.	Keep both configurations and switch, or use variables that are set per-network.
<code>git clone</code> hangs at 0%	Proxy not configured for git, or SSH blocked.	Listing C.1; or clone over HTTPS.
pip works, conda does not	conda reads its own config, not the environment variables.	<code>~/ .condarc</code> as above.
Docker build cannot resolve DNS	The daemon has its own network configuration.	<code>~/ .docker/config.json</code> ; restart Docker.
Certificate error only inside a container	The root certificate is on the host, not in the image.	<code>COPY</code> the certificate into the image and update its trust store.
A model download stalls at 90% repeatedly	Proxy buffering large responses.	Resume — most tools do; or download on another network and share the cache.

End of the manual.

Thirty-one golden rules, twenty-nine chapters and one small tool that you built from an empty folder to a tested, containerised, continuously integrated program. Keep this book near the machine. Write in it — the margins are where your own machine's quirks belong, and those notes will be worth more to you than anything printed here.

REVISION 1.0
17 MAY 2026